A.B.V.M. Agrawal Jatiya Kosh's

Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

DEPARTMENT OF INFORMATION TECHNOLOGY

Full Stack development MERN Practical

B.Sc. Information Technology

Submitted by

Ayesha Sarguru

Seat Number: 502520019

Under the Guidance of

Prof. Raviprakash Jaiswal

Academic Year 2026–27

October 2026

Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

Certificate by the Institute

Department of Information Technology

I hereby certify that Mr. **Ayesha Sarguru**, Seat Number 502520019, a student of Ghanshyamdas Jalan College of Science & Commerce, Malad (East), studying in **B.Sc. Information Technology**, has successfully completed the practical work in **Full Stack development MERN** for the academic year 2026–27, under the guidance of Prof. Raviprakash Jaiswal. To the best of my knowledge, the work of the student is original and the information included in the practical is correct.

Internal Guide	External Examiner	Head of the Department

Date: _____ **College Seal:**

INDEX

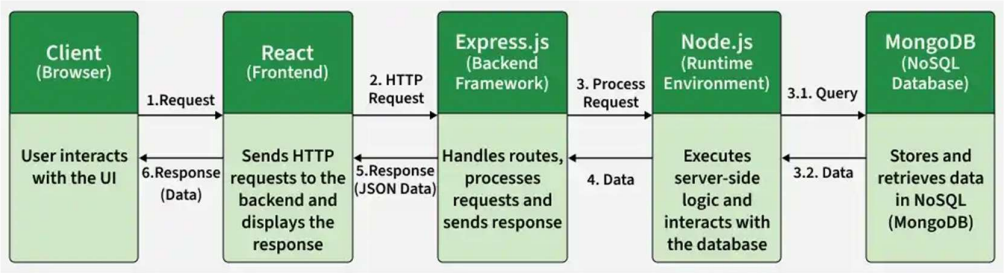
Sr. No.	Title	Page No.
1	**Module 1: Frontend Development using React** Environment Setup and JavaScript Basics	6-9
2	Core JavaScript Concepts for MERN	10-13
3	React Introduction and JSX	14-18
4	React Components, Props, and State	19-25
5	React Routing and Data Handling	26-31
6	**Module 2: Backend Development** Node.js and Express Fundamentals	32-38
7	MongoDB Basics	39-49
8	API Creation and Integration	50-64
9	User Data Management	65-78
10	Full Stack Execution and Validation	79-94

Full Stack Development using MERN means building both the **frontend (client-side)** and **backend (server-side)** of a web application using the **MERN Stack**.

MERN stands for:

- **M – MongoDB:** NoSQL database used to store application data.
- **E – Express.js:** Web framework for Node.js used to create APIs and handle server-side requests.
- **R – React:** JavaScript library used to build interactive user interfaces.
- **N – Node.js:** JavaScript runtime that executes server-side code.

MERN Stack Architecture



How a MERN Application Works

1. A user interacts with the **React** user interface.
2. React sends a request to the backend using HTTP (GET, POST, PUT, DELETE).
3. **Express.js** receives the request and processes it.
4. **Node.js** executes the server-side logic.
5. Data is stored in or retrieved from **MongoDB**.
6. The backend sends the response back to React.
7. React updates the user interface with the new data.

Features of MERN Stack

- Uses **JavaScript** throughout the application.
- Supports fast and responsive user interfaces.
- Enables easy communication through REST APIs.
- Suitable for scalable and modern web applications.
- Open-source with a large developer community.

Advantages

- One programming language (JavaScript) for frontend and backend.
- Faster development due to reusable code.
- Easy integration between technologies.
- Flexible document-based database (MongoDB).
- Excellent for Single Page Applications (SPAs).

Applications Built with MERN

- E-commerce websites
- Social media platforms
- Online learning portals
- Blogging websites
- Chat and messaging applications
- Task management systems

Skills Required

- HTML, CSS, JavaScript (ES6+)
- React.js
- Node.js
- Express.js
- MongoDB
- REST APIs
- Git and GitHub
- Authentication (JWT)
- Deployment (e.g., Vercel, Render, or similar services)

Full Stack Development using MERN is the process of developing both the frontend and backend of a web application using **MongoDB** for the database, **Express.js** for the backend framework, **React** for the frontend, and **Node.js** as the server-side runtime. It enables developers to build complete, dynamic, and scalable web applications using JavaScript across the entire stack.

Module 1: Frontend Development using React

Practical 1: Environment Setup and JavaScript Basics (Frontend Development using React)

Aim: To install the required software, understand JavaScript basics, and execute JavaScript programs using Node.js.

(a) Install Node.js and Visual Studio Code, and Verify Installation

Steps:

1. Download and install **Node.js** from the official website.
2. Install **Visual Studio Code (VS Code)**.
3. Open **Command Prompt** or **Terminal**.
4. Verify Node.js installation by typing:

```
node -v
```

5. Verify npm installation:

```
npm -v
```

6. If both commands display version numbers, the installation is successful.

Expected Output:

```
v22.x.x
10.x.x
(Version numbers may vary.)
```

(b) Create a Simple Node.js File and Run it Using the Terminal

Steps:

1. Open Visual Studio Code.
2. Create a new folder named **JavaScript_Practical**.
3. Create a file named **app.js**.
4. Type the following code:

app.js

```
console.log("Hello, Node.js!");
```

5. Save the file.
6. Open the terminal in VS Code.
7. Navigate to the project folder.
8. Run the program:

```
node app.js
```

Output:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical1\Pract1b> node app.js
Hello, Node.js!
Ayesha Sarguru
```

(c) Write a Node.js Script Demonstrating Basic JavaScript Commands

Steps:

1. Create a file named **basic.js**.
2. Write the following code:

basic.js

```
console.log("Basic JavaScript");
let a = 20;
let b = 10;
console.log("Value of a =", a);
console.log("Value of b =", b);
console.log("Addition =", a + b);
console.log("Subtraction =", a - b);
console.log("Multiplication =", a * b);
console.log("Division =", a / b);
```

3. Save the file.
4. Run the program:

```
node basic.js
```

Output:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical1> node basic.js
Basic JavaScript
Value of a = 20
Value of b = 10
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2
Ayesha Sarguru
```

(d) Create and Run a Simple Program Using Variables and Functions

Steps:

1. Create a file named **function.js**.
2. Write the following code:

function.js

```
let name = " Ayesha Sarguru ";
function greet(user) {
    return "Welcome " + user;
}
console.log(greet(name));
```

3. Save the file.
4. Run the program:

```
node function.js
```

Output:



(e) Demonstrate Use of Operators and Control Statements in JavaScript

Steps:

1. Create a file named **operators.js**.
2. Write the following code:

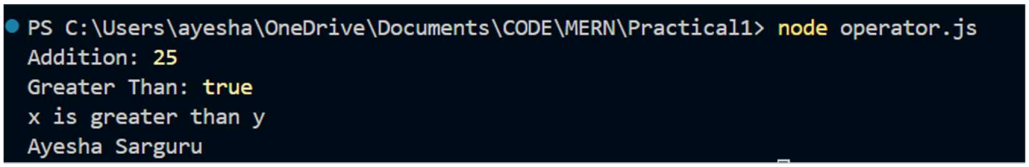
operators.js

```
let x = 15;
let y = 10;
console.log("Addition:", x + y);
console.log("Greater Than:", x > y);
if (x > y) {
    console.log("x is greater than y");
} else {
    console.log("y is greater than x");
}
```

3. Save the file.
4. Run the program:

```
node operators.js
```

Output:



(f) Write Programs Using Loops and Template Literals

Program 1: Using a For Loop

Steps:

1. Create a file named **loop.js**.

2. Write the following code:

loop.js

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
console.log("Ayesha Sarguru")
```

3. Save the file.
4. Run the program:

```
node loop.js
```

Output:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical1> node loop.js
1
2
3
4
5
Ayesha Sarguru
```

Program 2: Using Template Literals

Steps:

1. Create a file named **template.js**.
2. Write the following code:

template.js

```
let name = "Ayesha Sarguru";
let age = 20;
console.log(`My name is ${name} and I am ${age} years old.`);
```

3. Save the file.
4. Run the program:

```
node template.js
```

Output:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical1> node template.js
My name is Ayesha and I am 20 years old.
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical1>
```

Practical 2: Core JavaScript Concepts for MERN

Aim: To understand and implement core JavaScript concepts such as functions, arrays, objects, arrow functions, array methods, and combining these concepts in a JavaScript program.

(a) Create Functions and Call Them with Parameters

Steps:

1. Open Visual Studio Code.
2. Create a new file named **functions.js**.
3. Write the following code:

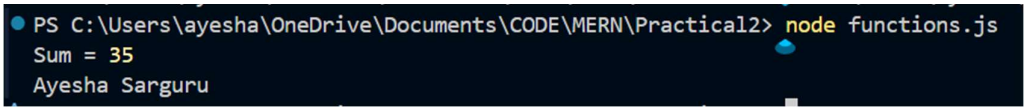
functions.js

```
function add(a, b) {
    return a + b;
}
let result = add(15, 20);
console.log("Sum =", result);
```

4. Save the file.
5. Open the terminal.
6. Run the program:

```
node functions.js
```

Output:



(b) Work with Arrays and Objects in JavaScript

Steps:

1. Create a file named **array_object.js**.
2. Write the following code:

array_object.js

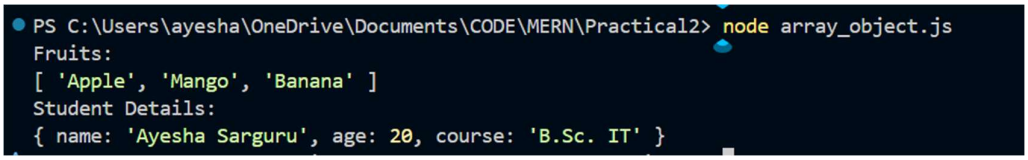
```
let fruits = ["Apple", "Mango", "Banana"];
console.log("Fruits:");
console.log(fruits);
let student = {
    name: "Ayesha Sarguru",
    age: 20,
    course: "B.Sc. IT"
};
console.log("Student Details:");
console.log(student);
```

3. Save the file.

4. Run the program:

```
node array_object.js
```

Output:



(c) Demonstrate Use of Arrow Functions

Steps:

1. Create a file named **arrow.js**.
2. Write the following code:

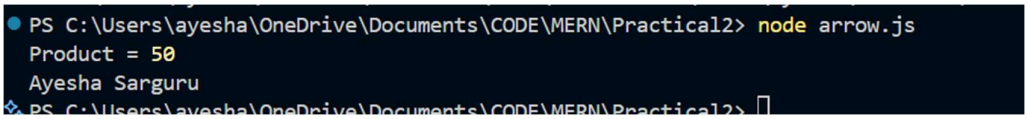
arrow.js

```
const multiply = (a, b) => a * b;
console.log("Product =", multiply(10, 5));
console.log("Ayesha Sarguru")
```

3. Save the file.
4. Run the program:

```
node arrow.js
```

Output:



(d) Write a Program Using Array Methods (map, filter, reduce)

Steps:

1. Create a file named **array_methods.js**.
2. Write the following code:

array_methods.js

```
let numbers = [10, 20, 30, 40, 50];
let doubled = numbers.map(num => num * 2)
let greaterThan25 = numbers.filter(num => num > 25);
let total = numbers.reduce((sum, num) => sum + num, 0);
console.log("Original Array:", numbers);
console.log("Mapped Array:", doubled);
console.log("Filtered Array:", greaterThan25);
console.log("Sum:", total);
console.log("Ayesha Sarguru")
```

3. Save the file.

4. Run the program:

```
node array_methods.js
```

Output:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical2> node array_methods.js
Original Array: [ 10, 20, 30, 40, 50 ]
Mapped Array: [ 20, 40, 60, 80, 100 ]
Filtered Array: [ 30, 40, 50 ]
Sum: 150
Ayesha Sarguru
```

(e) Create a Program that Manipulates Objects and Displays Output

Steps:

1. Create a file named **employee.js**.
2. Write the following code:

employee.js

```
let employee = {
    id: 101,
    name: "Ayesha Sarguru",
    department: "IT",
    salary: 50000
};
employee.salary = 55000;
employee.city = "Mumbai";
console.log("Employee Details");
console.log(employee);
```

3. Save the file.
4. Run the program:

```
node employee.js
```

Output:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\MERN\Practical2> node employee.js
Employee Details
{
  id: 101,
  name: 'Ayesha Sarguru',
  department: 'IT',
  salary: 55000,
  city: 'Mumbai'
}
```

(f) Implement a Small JavaScript Program Combining Functions, Arrays, and Objects

Steps:

1. Create a file named **student.js**.

2. Write the following code:

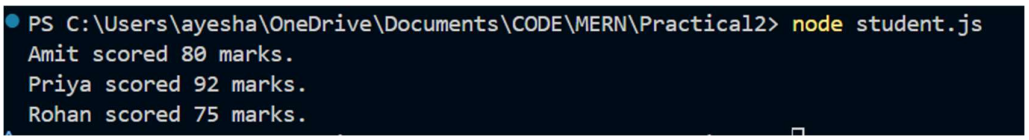
student.js

```
let students = [
    { id: 1, name: "Ayesha", marks: 80 },
    { id: 2, name: " Amit ", marks: 92 },
    { id: 3, name: "Rohan", marks: 75 }
];
function displayStudents(studentList) {
    studentList.forEach(student => {
        console.log(`${student.name} scored ${student.marks} marks.`);
    });
}
displayStudents(students);
```

3. Save the file.
4. Run the program:

```
node student.js
```

Output:

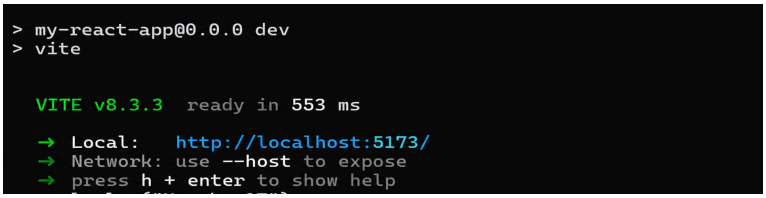


Practical 3: React Introduction and JSX

Aim: To create a React application and understand the basics of JSX, functional components, variables, functions, CSS styling, and dynamic content in React.

(a) Create a React Application Using Vite and Display "Welcome to MERN Stack Development"

Output:





(b) Display Dynamic Data Using JSX

Steps:

1. Open **src/App.jsx**.
2. Write the following code:

src/App.jsx

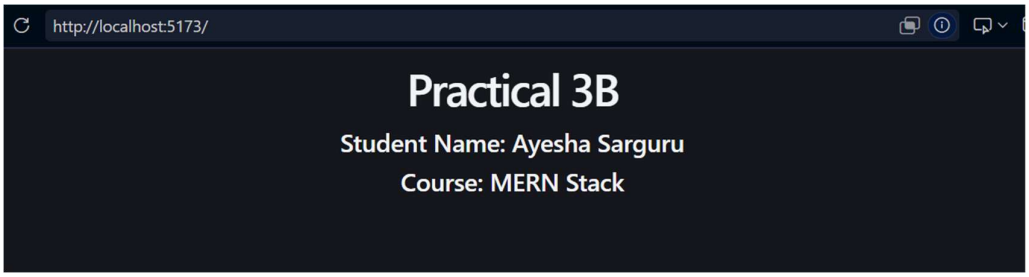
```
function App() {

  const name = "Ayesha Sarguru";
  const course = "MERN Stack";

  return (
    <>
      <h1>Practical 3B</h1>
      <h2>Student Name: {name}</h2>
      <h2>Course: {course}</h2>
    </>
  );
}
export default App;
```

3. Save the file.

Output:



(c) Declare Variables and Write a Function to Calculate the Sum of Two Numbers

Steps:

1. Open **src/App.jsx**.
2. Replace the code with:

src/App.jsx

```
function App() {
  let num1 = 15;
  let num2 = 25;
  function add(a, b) {
    return a + b;
  }
  return (
    <>
      <h1>Practical 3C</h1>
      <h2>Name: Ayesha Sarguru</h2>
      <h2>First Number: {num1}</h2>
      <h2>Second Number: {num2}</h2>
      <h2>Sum = {add(num1, num2)}</h2>
    </>
  );
}
export default App;
```

3. Save the file.

Output:



(d) Apply Basic CSS Styling to a React Component

Steps:

1. Open **src/App.css**.

2. Write the following CSS:

src/App.css

```
.container {
  text-align: center;
  color: blue;
  background-color: lightyellow;
  padding: 20px;
  border-radius: 10px;
  font-family: Arial;
}
```

3. Open **src/App.jsx**.

src/App.jsx

```
import "./App.css";
function App() {
  return (
    <h1>Practical 3D</h1>
    <h2>Ayesha Sarguru</h2>
    <h1>Welcome to React</h1>
    <p>Learning CSS in React</p>
    </div>
  );
}
export default App;
```

4. Save both files.

Output:



(e) Create a Simple Functional Component and Render It in App.jsx

Steps:

1. Create a folder named **components** inside the **src** folder.
2. Create a file named **Welcome.jsx**.

Welcome.jsx

```
function Welcome() {
  return (
    <h2>Welcome to Functional Components</h2>
  );
}
export default Welcome;
```

3. Open **App.jsx**.

App.jsx

```
import Welcome from "./components/Welcome";
function App() {
  return (
    <>
      <h1>Practical 3E</h1>
      <h2>Ayesha Sarguru</h2>
      <h1>React Components</h1>
      <Welcome />
    </>
  );
}
export default App;
```

4. Save the files.

Output:



(f) Create a React Component that Displays the Current Date and Time Dynamically Using JavaScript

Steps:

1. Open **src/App.jsx**.
2. Write the following code:

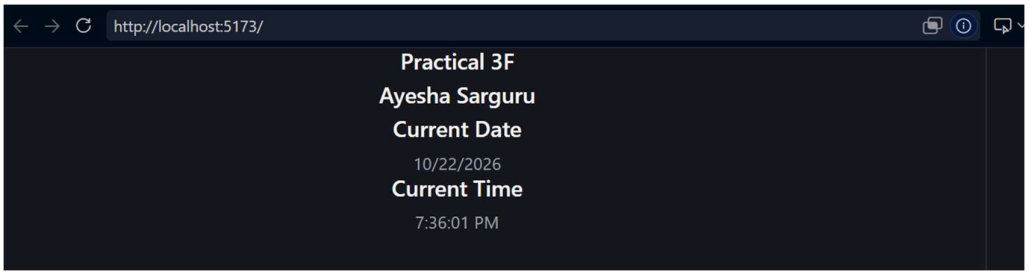
src/App.jsx

```
function App() {
  const today = new Date();
  return (
    <>
      <h2>Current Date</h2>
      <p>{today.toLocaleDateString()}</p>

      <h2>Current Time</h2>
      <p>{today.toLocaleTimeString()}</p>
    </>
  );
}
export default App;
```

3. Save the file.
4. Refresh the browser to see the updated date and time.

Output (Example):



Result

The React application was successfully created using Vite. Programs demonstrating JSX, dynamic data rendering, variables, functions, CSS styling, functional components, and displaying the current date and time were implemented and executed successfully.

Practical 4: React Components, Props, and State

Aim: To understand React components, props, state management using the useState hook, forms, list rendering with map(), and build interactive React components.

(a) Create Multiple Components and Render Them on a Single Page

Steps:

1. Open the React project created using Vite.
2. Inside the **src** folder, create a folder named **components**.
3. Create two component files: **Header.jsx** and **Footer.jsx**.

Header.jsx

```
function Header() {
  return <h1>Welcome to MERN Stack Development</h1>;
}
export default Header;
```

Footer.jsx

```
function Footer() {
  return <h3>Thank You for Visiting</h3>;
}
export default Footer;
```

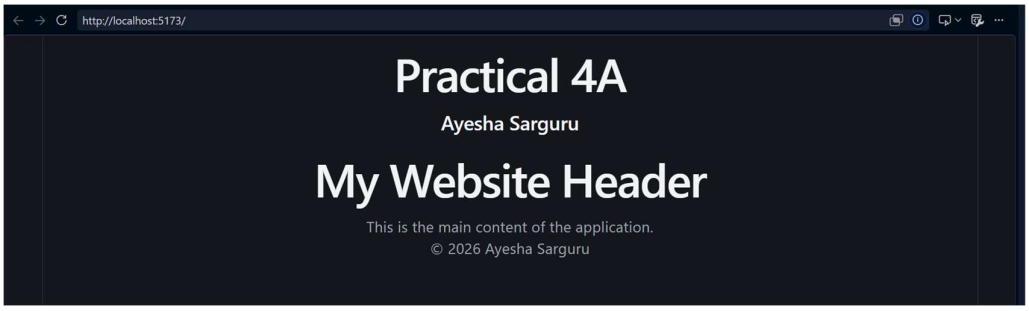
4. Open **App.jsx** and write:

App.jsx

```
import Header from "./components/Header";
import Footer from "./components/Footer";
function App() {
  return (
    <>
      <h1>Practical 4A</h1>
      <h2>Ayesha Sarguru</h2>
      <Header />
      <p>This is the main content of the application.</p>
      <Footer />
    </>
  );
}
export default App;
```

5. Save the files and run the application.

Output:



(b) Pass Data Between Components Using Props

Steps:

1. Create **Student.jsx** inside the **components** folder.

Student.jsx

```
function Student(props) {
  return (
    <>
      <h2>Name: {props.name}</h2>
      <h3>Course: {props.course}</h3>
    </>
  );
}
export default Student;
```

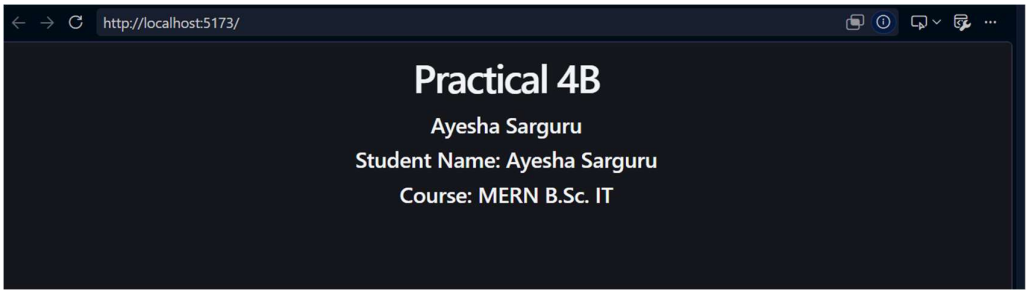
2. Open **App.jsx**.

App.jsx

```
import Student from "./components/Student";
function App() {
  return (
    <>
      <h1>Practical 4A</h1>
      <h2>Ayesha Sarguru</h2>
      <Header />
      <p>This is the main content of the application.</p>
      <Footer />
    </>
  );
}
export default App;
```

3. Save and run the project.

Output:



(c) Use the useState Hook to Manage and Update Data

Steps:

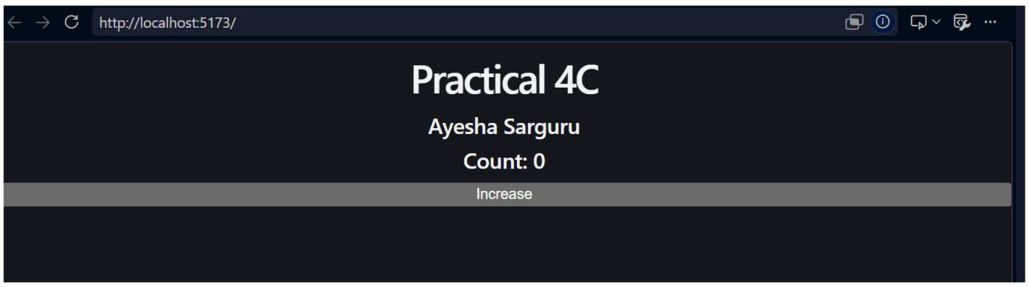
1. Open **App.jsx**.
2. Write the following code:

App.jsx

```
import { useState } from "react";
function App() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h2>Counter: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </>
  );
}
export default App;
```

3. Save the file.
4. Click the **Increment** button to update the count.

Output:





(d) Create a Form Component to Accept User Input and Display It Dynamically

Steps:

1. Open **App.jsx**.
2. Write the following code:

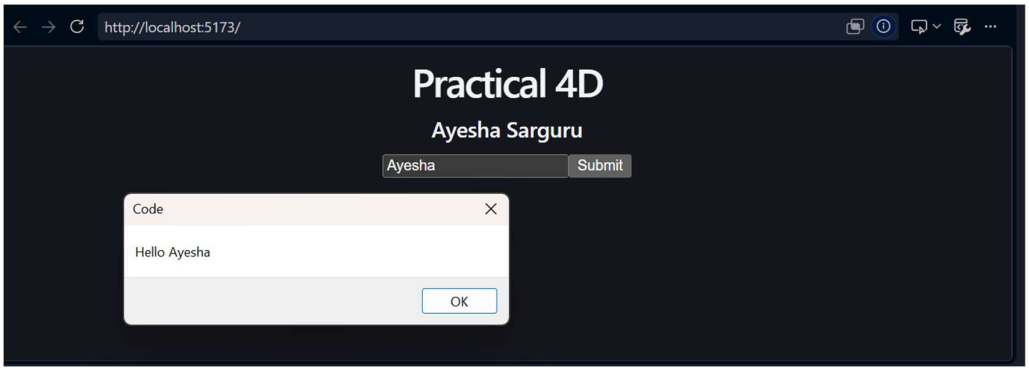
App.jsx

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");
  return (
    <>
      <h2>Student Form</h2>
      <input
        type="text"
        placeholder="Enter Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <h3>You entered: {name}</h3>
    </>
  );
}
export default App;
```

3. Save the file.
4. Enter text in the input box.

Output:



(e) Display a List of Items Stored in the State Using map()

Steps:

1. Open **App.jsx**.

Write the following code:

App.jsx

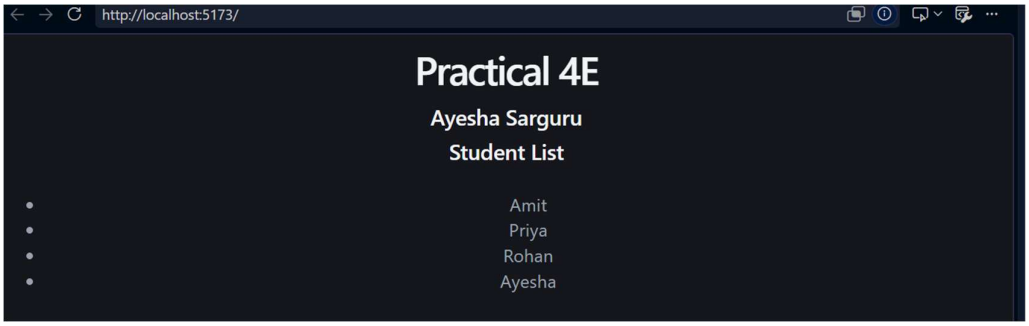
```
function App() {
  const students = ["Amit", "Priya", "Rohan", "Ayesha"];

  return (
    <>
      <h1>Practical 4E</h1>
      <h2>Ayesha Sarguru</h2>

      <h2>Student List</h2>

      <ul>
        {students.map((student, index) => (
          <li key={index}>{student}</li>
        ))}
      </ul>
    </>
  );
}
export default App;
```

Output:



(f) Build a Small Interactive Component (Counter)

Steps:

1. Open **App.jsx**.

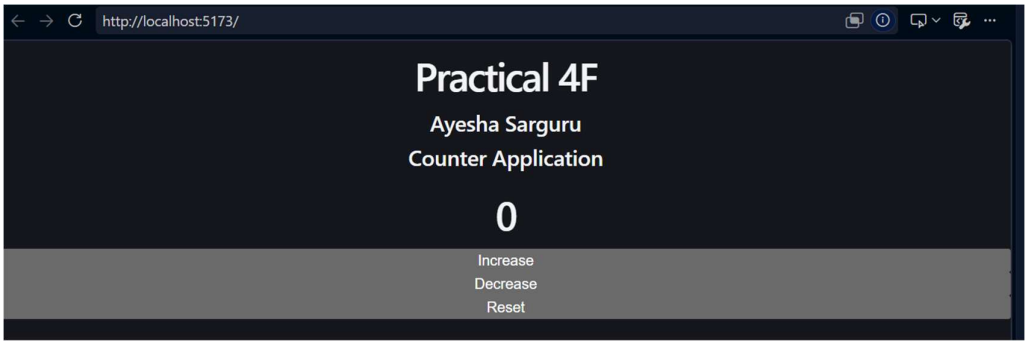
2. Write the following code:

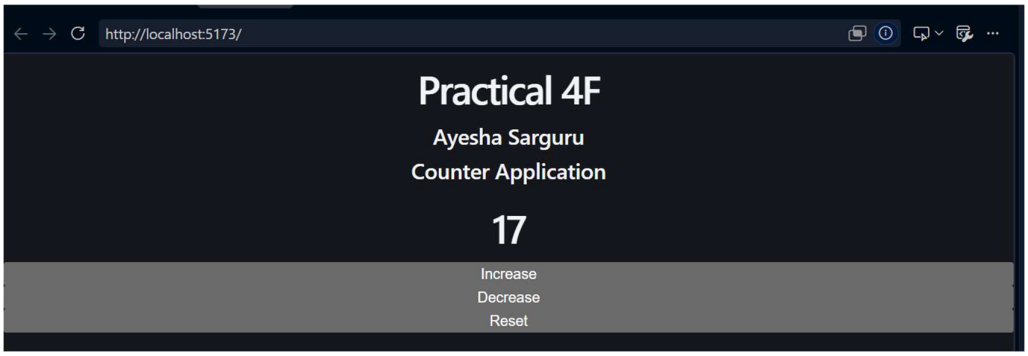
App.jsx

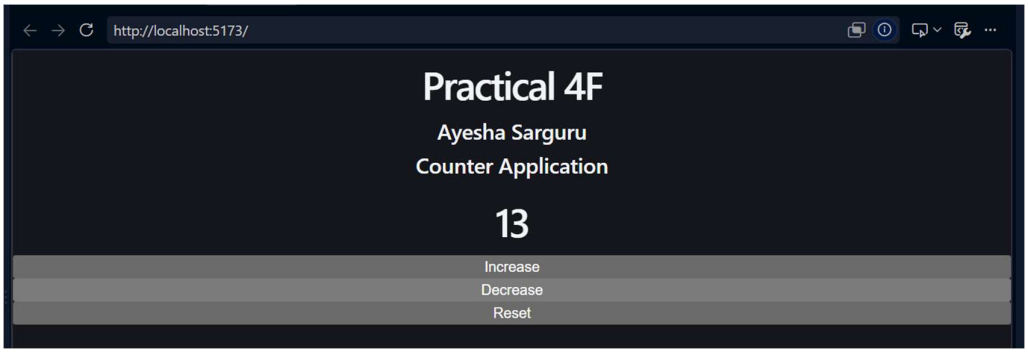
```
import { useState } from "react";
function App() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h2>Counter Application</h2>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
      <button onClick={() => setCount(count - 1)}>
        Decrease
      </button>
      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </>
  );
}
export default App;
```

3. Save the file.
4. Click the buttons to increase, decrease, and reset the counter.

Output:







Result

The practical was completed successfully. Multiple React components were created and rendered, data was passed using props, state was managed using the useState hook, user input was accepted through a form, lists were displayed using map(), and an interactive counter application was implemented successfully.

Practical 5: React Routing and Data Handling

Aim: To understand React Router, navigation between pages, fetching data from a public API using Axios, displaying data, and implementing search/filter functionality.

(a) Install and Configure React Router in the React Application

Steps:

1. Open the React project in Visual Studio Code.
2. Open the integrated terminal.
3. Install **React Router DOM**

(b) Create Multiple Pages and Implement Navigation Between Them

Steps:

1. Create a folder named **pages** inside the **src** folder.
2. Create three files:

Home.jsx

```
function Home() {
  return <h2>Welcome to Home Page</h2>;
}
export default Home;
```

About.jsx

```
function About() {
  return <h2>Welcome to About Page</h2>;
}
export default About;
```

Contact.jsx

```
function Contact() {
  return <h2>Welcome to Contact Page</h2>;
}
export default Contact;
```

(c) Use Link and Routes to Move Between Pages

Steps:

1. Open **App.jsx**.

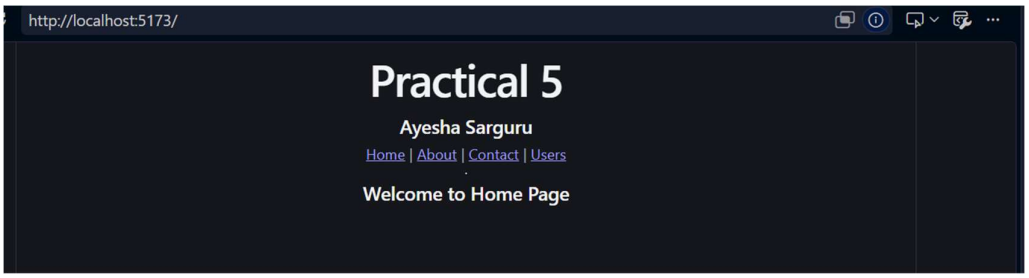
2. Write the following code:

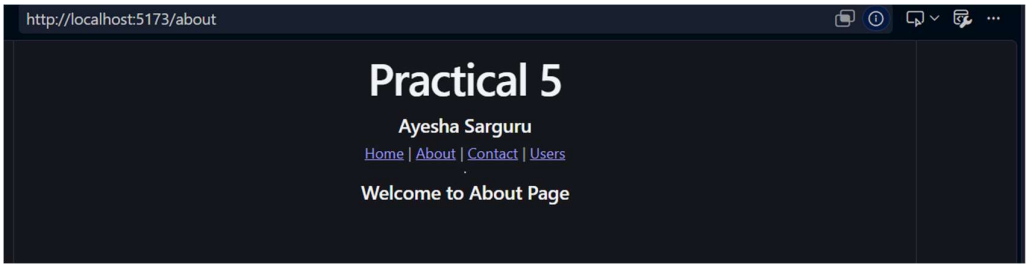
App.jsx

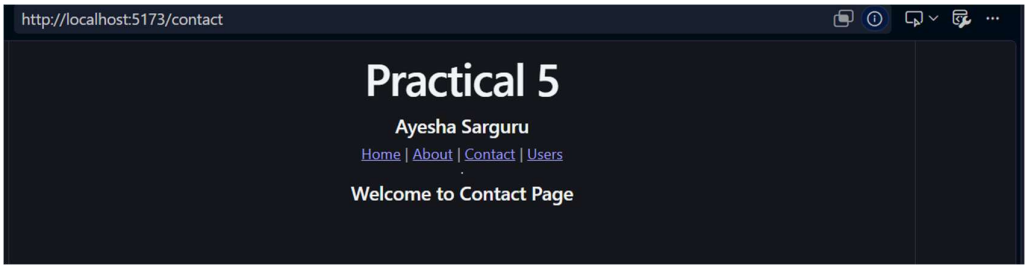
```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";
function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> |{" "}
      <Link to="/about">About</Link> |{" "}
    <Link to="/contact">Contact</Link>
    </nav>
    <hr />
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
    </BrowserRouter>
  );
}
export default App;
```

3. Save the file.
4. Run the application.

Output:







(d) Fetch Sample Data from a Public API Using Axios

Steps:

1. Install Axios:

```
npm install axios
```

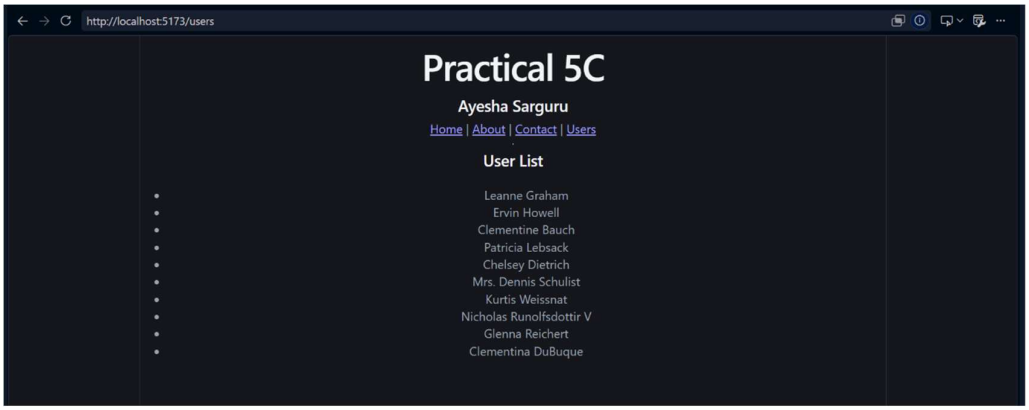
2. Create a file named **Users.jsx** inside the **pages** folder.
3. Write the following code:

Users.jsx

```
import axios from "axios";
import { useEffect, useState } from "react";
function Users() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      });
  }, []);
  return (
    <>
      <h2>User List</h2>
      <ul>
        {users.map((user) => (
          <li key={user.id}>
            {user.name}
          </li>
        ))}
      </ul>
    </>
  );
}
export default Users;
```

4. Save the file.

Output:



(e) Display Fetched Data in Table Format

Steps:

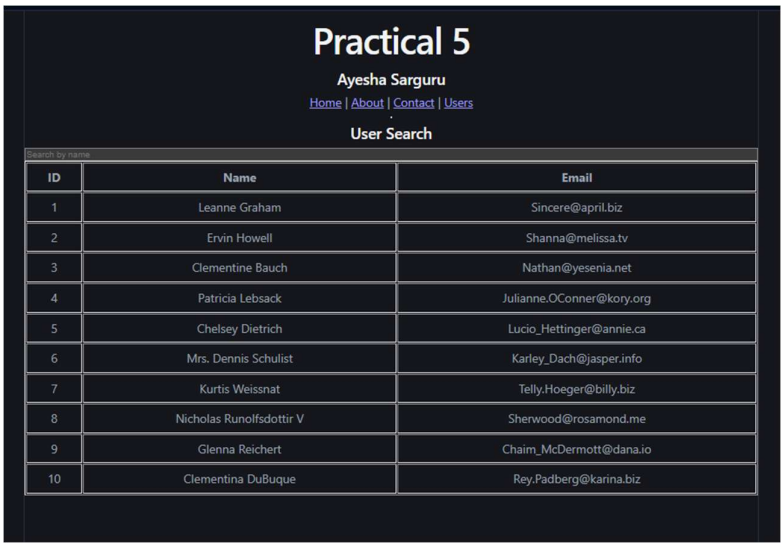
1. Modify **Users.jsx** as follows:

Users.jsx

```
import axios from "axios";
import { useEffect, useState } from "react";
function Users() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => setUsers(response.data));
  }, []);
  return (
    <>
      <h2>User Details</h2>
      <table border="1" cellPadding="8">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  );
}
export default Users;
```

3. Save the file.

Output:



(f) Implement Search or Filter Functionality on Displayed Data

Steps:

1. Update **Users.jsx** with the following code:

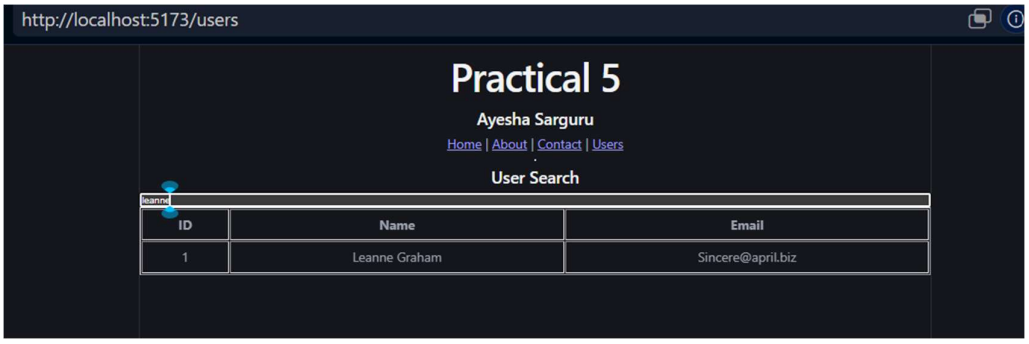
Users.jsx

```
import axios from "axios";
import { useEffect, useState } from "react";
function Users() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");
  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => setUsers(response.data));
  }, []);
  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );
  return (
    <>
      <h2>User Search</h2>
      <input
        type="text"
        placeholder="Search by name"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
      <table border="1" cellPadding="8">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
          </tr>
        </thead>
        <tbody>
```

```
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  );
}
export default Users;
```

2. Save the file.

Output:



Result

The practical was completed successfully. React Router was installed and configured, multiple pages were created and connected using BrowserRouter, Routes, Route, and Link. Data was fetched from a public API using Axios, displayed in a table, and a search/filter feature was implemented using React state and array filtering.

Module 2: Backend Development using Node.js, Express, and MongoDB

Practical 6: Node.js and Express Fundamentals

Aim: To install Express.js, create a simple web server using Node.js, create different routes, send text and JSON responses, and test the routes using a web browser or Postman.

(a) Install Express and create a simple server

Part A: Create a Node.js Project

Step 1: Create a Project Folder

Open Command Prompt or Terminal:

```
mkdir ExpressDemo
cd ExpressDemo
```

Step 2: Initialize Node.js Project

Execute:

```
npm init -y
```

Part B: Install Express

Execute:

```
npm install express
```

Part C: Create a Simple Express Server

Step 1: Create app.js

Inside the ExpressDemo folder, create:

```
app.js
```

Step 2: Write the following program

app.js

```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/", (req, res) => {
    res.send("Welcome to Node.js and Express!");
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```



Part D: Understand the Program

1. Import Express

```
const express = require("express");
```

or

```
import express from "express";
```

This imports the Express.js module into the application.

2. Create Express Application

```
const app = express();
```

The app object is used to configure the Express server and define routes.

3. Define Port

```
const PORT = 3000;
```

The server will listen for requests on port 3000.

Therefore, the application can be accessed at:

```
http://localhost:3000
```

4. Create a Route

```
app.get("/", (req, res) => {
    res.send("Welcome to Node.js and Express!");
});
```

5. Start the Server

```
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

This starts the Express server.

Part E: Run the Server

Open the terminal inside the project folder and execute:

```
node app.js
```

Part F: Display a Message in the Browser

Open a web browser and enter:

```
http://localhost:3000
```

Part G: Create Different Routes

Now create multiple routes in app.js.

Replace the previous code with:

app.js

```
const express = require("express");
const app = express();
const PORT = 3000;
// Home Route
app.get("/", (req, res) => {
    res.send("Welcome to Express Server!");
});
// About Route
app.get("/about", (req, res) => {
    res.send("This is the About Page.");
});
// Contact Route
app.get("/contact", (req, res) => {
    res.send("Contact Us at contact@example.com");
});
// Students Route
app.get("/students", (req, res) => {
    res.send("Student Information Page");
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Part H: Test Different Routes

Start the server:

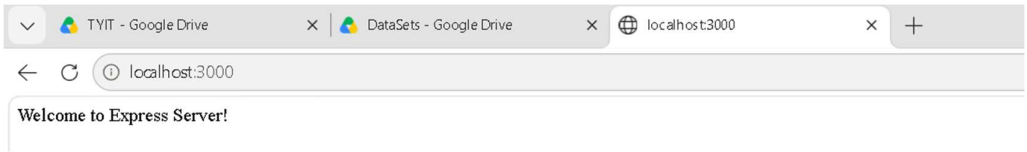
```
node app.js
```

Now test the following URLs.

1. Home Route

```
http://localhost:3000/
```

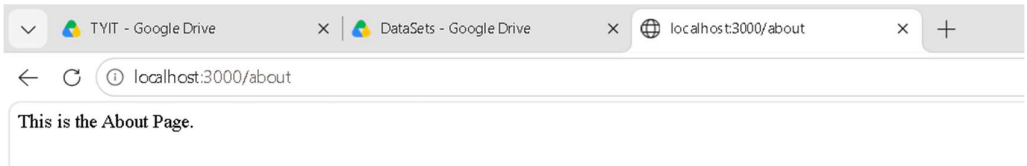
Output:



2. About Route

```
http://localhost:3000/about
```

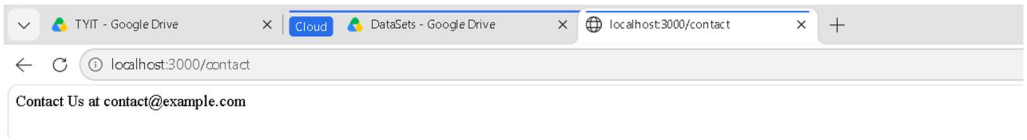
Output:



3. Contact Route

```
http://localhost:3000/contact
```

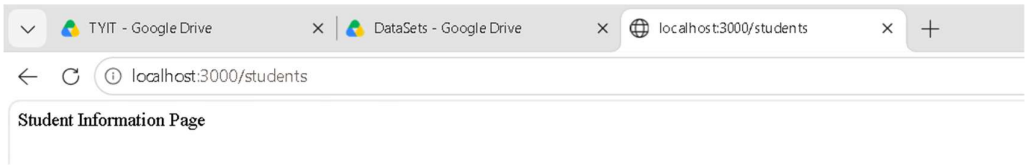
Output:



4. Students Route

```
http://localhost:3000/students
```

Output:



Part I: Send a JSON Response

Express can send data in JSON format using:

```
res.json()
```

Add the following route:

```
app.get("/api/student", (req, res) => {

    res.json({
        rollNo: 101,
        name: "Rahul",
        course: "BSc IT",
        semester: 5,
        marks: 85
    });
});
```

The complete program becomes:

```
const express = require("express");
const app = express();
const PORT = 3000;
// Home Route
app.get("/", (req, res) => {
    res.send("Welcome to Express Server!");
});
// About Route
app.get("/about", (req, res) => {
    res.send("This is the About Page.");
});
// Contact Route
app.get("/contact", (req, res) => {
    res.send("Contact Us at contact@example.com");
});
// Student API Route
app.get("/api/student", (req, res) => {
    res.json({
        rollNo: 101,
        name: "Ayesha",
        course: "BSc IT",
        semester: 5,
        marks: 85
    });
});
// Start Server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Part J: Test JSON Route in Browser

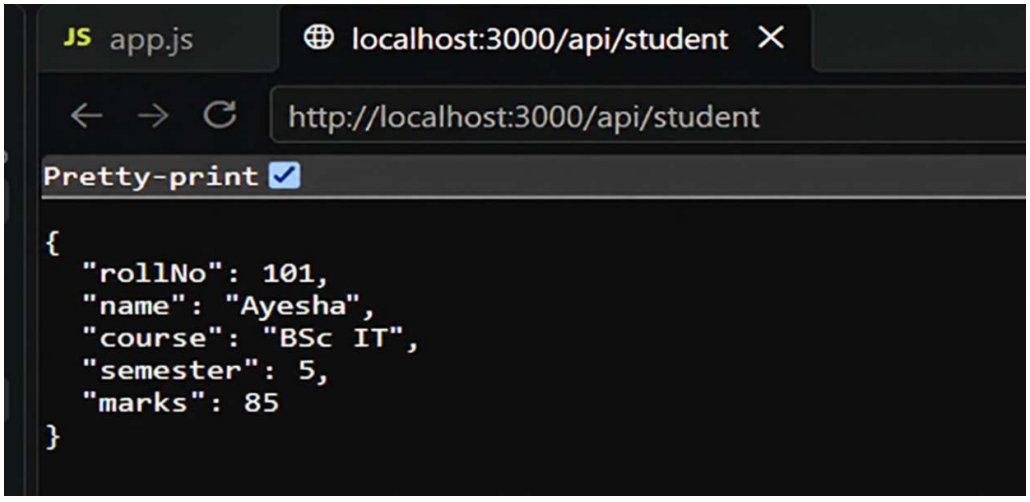
Run:

```
node app.js
```

Open:

```
http://localhost:3000/api/student
```

This demonstrates how Express can be used to create a simple REST API.



Part K: Send Multiple Student Records as JSON

Create another route:

```
app.get("/api/students", (req, res) => {
```

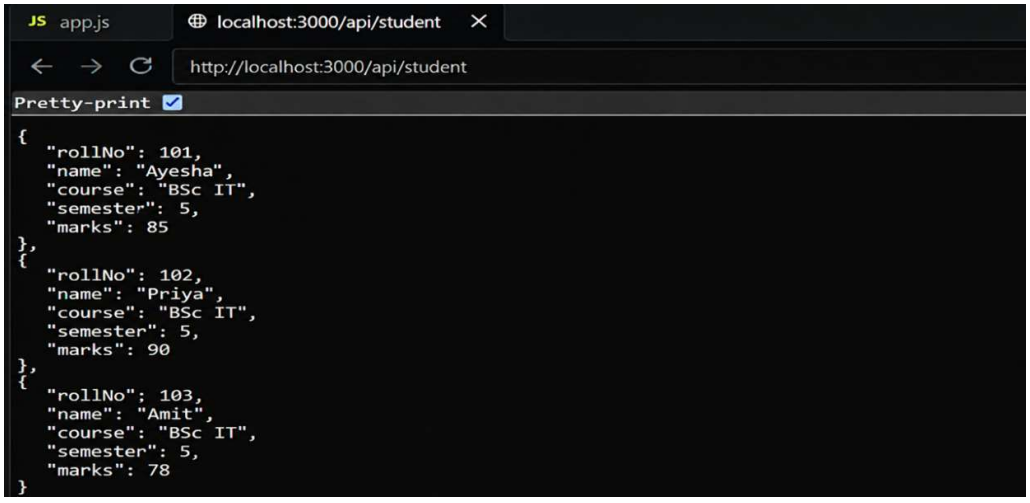
Test:

```
http://localhost:3000/api/students
```

Expected Response:

```
[
    {
        "rollNo": 101,
        "name": "Ayesha",
        "course": "BSc IT",
        "marks": 85
    },
    {
        "rollNo": 102,
        "name": "Priya",
        "course": "BSc IT",
        "marks": 90
    },
    {
        "rollNo": 103,
        "name": "Amit",
        "course": "BSc IT",
        "marks": 78
    }
]
```

Output:



Part L: Test Routes Using Postman

Postman can be used to test Express APIs.

Step 1: Start the Server

```
node app.js
```

Step 2: Open Postman

Create a new HTTP request.

Select:

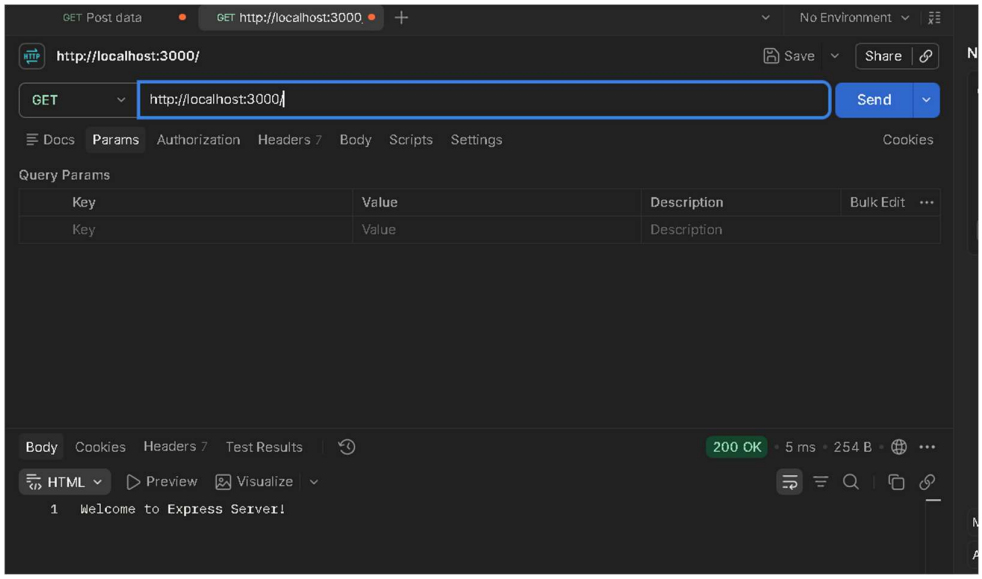
```
GET
```

Enter:

```
http://localhost:3000/
```

Click **Send**.

Response:



Result

Thus, Express.js was successfully installed and configured with Node.js. A simple Express server was created, multiple routes were implemented, text and JSON responses were generated, and the routes were successfully tested using a web browser/Postman. The server output was also observed in the terminal.

Practical 7: MongoDB Basics

Aim: To install and start MongoDB, create a database and collection, perform CRUD operations, and connect MongoDB with a Node.js application.

Part A: Install and Start MongoDB

Step 1: Install MongoDB

1. Download and install **MongoDB Community Server**.
2. During installation, select the option to install MongoDB as a **Windows Service** if available.
3. Complete the installation.
4. Verify the installation by opening Command Prompt and executing:

```
mongosh
```

If MongoDB Shell opens successfully, MongoDB is installed.

Step 2: Start MongoDB

If MongoDB is installed as a Windows service, it can be started using:

```
net start MongoDB
```

Then open MongoDB Shell:

```
mongosh
```

You should see a connection to the local MongoDB server, normally:

```
mongodb://127.0.0.1:27017/
Default MongoDB port: 27017
```

Part B: Create a Database and Collection

Step 1: Open MongoDB Shell

```
mongosh
```

Step 2: Create/Select a Database

Use the use command:

```
use CollegeDB
```

Output:

```
mongosh> use CollegeDB
switched to db CollegeDB
CollegeDB>
```

Step 3: Create a Collection

Create a collection named students:

```
db.createCollection("students")
```

Output:

```
CollegeDB> db.createCollection("students")
{ ok: 1 }
```

Or

```
db.createCollection("students", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["rollNo", "name", "course", "semester", "marks"],
            properties: {
                rollNo: {
                    bsonType: "int",
                    description: "Roll number must be an integer"
                },
                name: {
                    bsonType: "string",
                    description: "Name must be a string"
                },
                course: {
                    bsonType: "string",
                    description: "Course must be a string"
                },
                semester: {
                    bsonType: "int",
                    minimum: 1,
                    maximum: 6,
                    description: "Semester must be between 1 and 6"
                },
                marks: {
                    bsonType: "int",
                    minimum: 0,
                    maximum: 100,
                    description: "Marks must be between 0 and 100"
                }
            }
        }
    }
})
```

Step 4: Display Collections

```
show collections
```

Output:

```
{ ok: 1 }
CollegeDB> show collections
students
```

Part C: Insert Records into MongoDB

MongoDB stores data in the form of **documents**, which are similar to JSON objects.

1. Insert One Record

```
db.students.insertOne({
    rollNo: 101,
    name: "Rahul",
    course: "BSc IT",
    semester: 1,
    marks: 78
})
```

MongoDB automatically generates an _id for the document.

Example:

```
CollegeDB> db.students.insertOne({
|       rollNo: 20,
|       name: "Ayesha",
|       course: "BSc IT",
|       semester: 1,
|       marks: 99
|   })
|
{
  acknowledged: true,
  insertedId: ObjectId('6ac68c6ca1362209fe25b513')
```

2. Insert Multiple Records

```
db.students.insertMany([
    {
        rollNo: 102,
        name: "Ayesha",
        course: "BSc IT",
        semester: 1,
        marks: 85
    },
    {
        rollNo: 103,
        name: "Saamiya",
        course: "BSc IT",
        semester: 1,
        marks: 72
    },
    {
        rollNo: 104,
        name: "Piyush",
        course: "BSc IT",
        semester: 1,
        marks: 91
    }
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac68cb7a1362209fe25b514'),
    '1': ObjectId('6ac68cb7a1362209fe25b515'),
    '2': ObjectId('6ac68cb7a1362209fe25b516')
  }
}
```

Part D: View Records from the Collection

1. Display All Records

```
db.students.find()
```

```
college> db.students.find()
[
  {
    _id: ObjectId('6abc7c3b78ddfda3a7b5bbed'),
    rollNo: 20,
    name: 'Ayesha',
    course: 'BSc IT',
    semester: 1,
    marks: 99
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbee'),
    rollNo: 102,
    name: 'Priya',
    course: 'BSc IT',
    semester: 1,
    marks: 85
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbef'),
    rollNo: 103,
    name: 'Amit',
    course: 'BSc IT',
    semester: 1,
    marks: 72
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbf0'),
    rollNo: 104,
    name: 'Sneha',
    course: 'BSc IT',
    semester: 1,
    marks: 91
  }
]
college>
```

```
db.students.find().pretty()
```

2. Find a Particular Student

```
db.students.findOne({ rollNo: 102 })
```

```
]
college> db.students.findOne({ rollNo: 102 })
{
  _id: ObjectId('6abc7c6d78ddfda3a7b5bbee'),
  rollNo: 102,
  name: 'Priya',
  course: 'BSc IT',
  semester: 1,
  marks: 85
}
college>
```

3. Find Students with Marks Greater Than 80

```
db.students.find({ marks: { $gt: 80 } })
```

```
college> db.students.find({ marks: { $gt: 80 } })
[
  {
    _id: ObjectId('6abc7c3b78ddfda3a7b5bbed'),
    rollNo: 20,
    name: 'Ayesha',
    course: 'BSc IT',
    semester: 1,
    marks: 99
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbee'),
    rollNo: 102,
    name: 'Priya',
    course: 'BSc IT',
    semester: 1,
    marks: 85
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbf0'),
    rollNo: 104,
    name: 'Sneha',
    course: 'BSc IT',
    semester: 1,
    marks: 91
  }
]
college>
```

4. Find Students from BSc IT

```
db.students.find({ course: "BSc IT" })
```

```
college> db.students.find({ course: "BSc IT" })
[
  {
    _id: ObjectId('6abc7c3b78ddfda3a7b5bbed'),
    rollNo: 20,
    name: 'Ayesha',
    course: 'BSc IT',
    semester: 1,
    marks: 99
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbee'),
    rollNo: 102,
    name: 'Priya',
    course: 'BSc IT',
    semester: 1,
    marks: 85
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbef'),
    rollNo: 103,
    name: 'Amit',
    course: 'BSc IT',
    semester: 1,
    marks: 72
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbf0'),
    rollNo: 104,
    name: 'Sneha',
    course: 'BSc IT',
    semester: 1,
    marks: 91
  }
]
college>
```

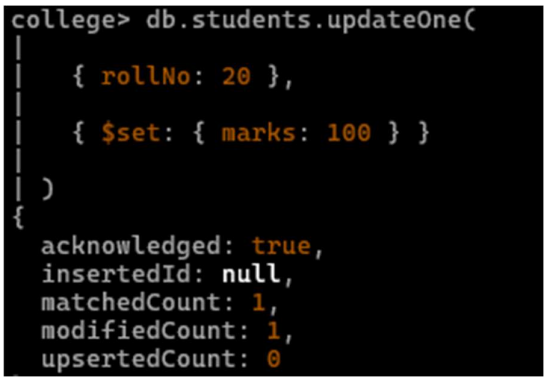
Part E: Update Records

MongoDB provides updateOne() and updateMany() for modifying documents.

1. Update One Record

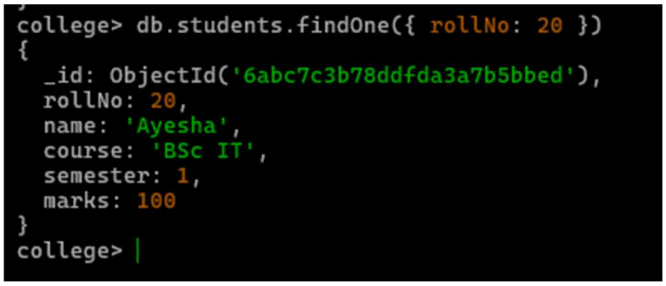
Suppose we want to change Ayesha's marks from 99 to 100:

```
db.students.updateOne(
    { rollNo: 20},
    { $set: { marks: 100 } }
)
```



Verify:

```
db.students.findOne({ rollNo: 20})
```



2. Update Multiple Records

For example, change the semester of all students:

```
db.students.updateMany(
    { course: "BSc IT" },
    { $set: { semester: 2 } }
)
```

Verify:

```
db.students.find()
```

```
college> db.students.find().pretty()
[
  {
    _id: ObjectId('6abc7c3b78ddfda3a7b5bbed'),
    rollNo: 20,
    name: 'Ayesha',
    course: 'BSc IT',
    semester: 2,
    marks: 100
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbee'),
    rollNo: 102,
    name: 'Priya',
    course: 'BSc IT',
    semester: 2,
    marks: 85
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbef'),
    rollNo: 103,
    name: 'Amit',
    course: 'BSc IT',
    semester: 2,
    marks: 72
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbf0'),
    rollNo: 104,
    name: 'Sneha',
    course: 'BSc IT',
    semester: 2,
    marks: 91
  }
]
college>
```

Part F: Delete Records

1. Delete One Record

Delete the student having roll number 104:

```
db.students.deleteOne({ rollNo: 104 })
```

```
college> db.students.deleteOne({ rollNo: 104 })
{ acknowledged: true, deletedCount: 1 }
college>
```

Verify:

```
db.students.find()
```

```
college> db.students.find()
[
  {
    _id: ObjectId('6abc7c3b78ddfda3a7b5bbed'),
    rollNo: 20,
    name: 'Ayesha',
    course: 'BSc IT',
    semester: 2,
    marks: 100
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbee'),
    rollNo: 102,
    name: 'Priya',
    course: 'BSc IT',
    semester: 2,
    marks: 85
  },
  {
    _id: ObjectId('6abc7c6d78ddfda3a7b5bbef'),
    rollNo: 103,
    name: 'Amit',
    course: 'BSc IT',
    semester: 2,
    marks: 72
  }
]
college>
```

2. Delete Multiple Records

For example:

```
db.students.deleteMany({ semester: 2 })
```

```
CollegeDB> db.students.deleteMany({ semester: 2 })
{ acknowledged: true, deletedCount: 3 }
```

Note: deleteMany() should be used carefully because it can remove multiple documents.

Part G: Connect MongoDB with Node.js

Now connect MongoDB with a Node.js application.

Step 1: Create a Node.js Project

Open Command Prompt or Terminal:

```
mkdir MongoDBNodeDemo
cd MongoDBNodeDemo
```

Initialize the Node.js project:

```
npm init -y
```

This creates:

```
package.json
```

```
C:\Users\dell\MongoDBNodeDemo>npm init -y
Wrote to C:\Users\dell\MongoDBNodeDemo\package.json:

{
  "name": "mongodbnodedemo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Step 2: Install MongoDB Driver

Install the official MongoDB Node.js driver:

```
npm install mongodb
```

The mongodb package allows Node.js applications to communicate with MongoDB.

Step 3: Create JavaScript File

Create a file:

```
app.js
```

Project structure:

```
MongoDBNodeDemo
│
├── node_modules
├── package.json
├── package-lock.json
└── app.js
```

Step 4: Write Node.js Connection Code

app.js

```
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";

const client = new MongoClient(url);

async function main() {
    try {
        await client.connect();

        console.log("Connected to MongoDB successfully!");

        const db = client.db("CollegeDB");

        const collection = db.collection("students");

        const students = await collection.find().toArray();

        console.log("Student Records:");

        console.log(students);
    }
    catch (error) {
        console.error("Error:", error);
    }
    finally {
        await client.close();
    }
}

main();
```

Step 5: Run the Node.js Application

Make sure MongoDB is running.

Execute:

```
node app.js
```

Expected Output:

```
Connected to MongoDB successfully!

Student Records:
```



Result

Thus, MongoDB was successfully installed and started. A database and collection were created, records were inserted and retrieved, and update and delete operations were performed successfully. Finally, MongoDB was connected with a Node.js application using the MongoDB Node.js driver.

Practical 8: API Creation and Integration

Aim: To create a REST API using Node.js and Express, retrieve data using a GET request, send data from a React application using a POST request, store the data in MongoDB, and display the stored data on a React webpage.

(a) REST API Creation and Integration using Express, React and MongoDB

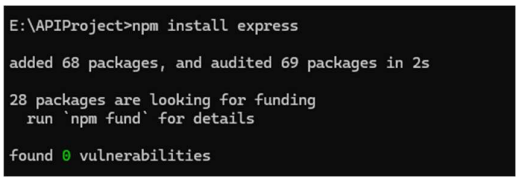
Part A: Create the Express REST API

Step 1: Create Backend Folder

Step 2: Install Required Packages

Install Express:

```
npm install express
```

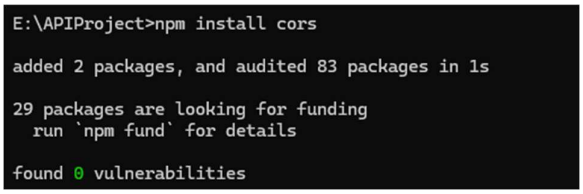


Install MongoDB driver:

```
npm install mongodb
```

Install CORS:

```
npm install cors
```



The project structure will be:

```
APIProject
│
├── node_modules
├── package.json
├── package-lock.json
└── server.js
```

Part B: Create the Express Server

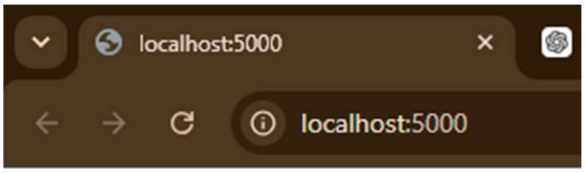
Create a file named:

```
server.js
```

Enter the following code:

server.js

```
const express = require("express");
const cors = require("cors");
const app = express();
const PORT = 5000;
app.use(cors());
app.use(express.json());
app.get("/", (req, res) => {
    res.send("Student REST API is running");
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```



Student REST API is running

Part C: Create a GET API

Add the following route to server.js:

server.js

```
const express = require("express");
const cors = require("cors");
const app = express();
const PORT = 5000;
// Middleware
app.use(cors());
app.use(express.json());
// Sample student data
const students = [
    {
        rollNo: 101,
        name: "Ayesha",
        course: "BSc IT",
        semester: 1,
        marks: 85
    },
    {
        rollNo: 102,
        name: "Priya",
        course: "BSc IT",
        semester: 1,
        marks: 90
    },
    {
        rollNo: 103,
        name: "Amit",
        course: "BSc IT",
        semester: 1,
        marks: 78
    }
];
```

```
// Home route
app.get("/", (req, res) => {
    res.send("Student REST API is running");
});
// GET API
app.get("/api/students", (req, res) => {
    res.json(students);
});
// Start server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Part D: Run and Test the GET API

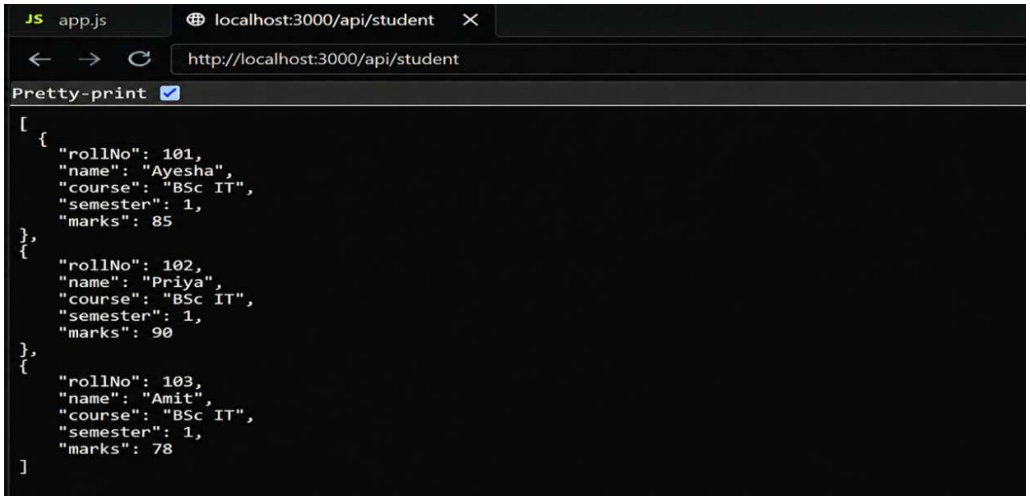
Start the server:

```
node server.js
```

Expected console output:

```
Server running at http://localhost:5000
Open the browser:
http://localhost:5000/api/students
```

Expected Output:



Part E: Create React Application

Open another terminal.

Create a React application:

```
npm create vite@latest student-client
```

Part F: Fetch Data from API in React

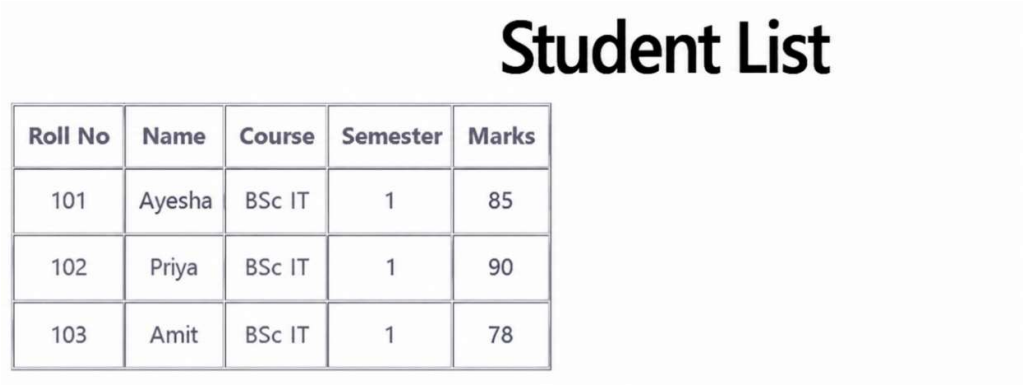
Open:

```
src/App.jsx
```

Replace the existing code with:

src/App.jsx

```
import { useEffect, useState } from "react";
function App() {
    const [students, setStudents] = useState([]);
    useEffect(() => {
        fetch("http://localhost:5000/api/students")
            .then(response => response.json())
            .then(data => {
                setStudents(data);
            })
            .catch(error => {
                console.error("Error fetching data:", error);
            });
    }, []);
    return (
        <div>
            <h1>Student List</h1>
            <table border="1" cellPadding="10">
                <thead>
                    <tr>
                        <th>Roll No</th>
                        <th>Name</th>
                        <th>Course</th>
                        <th>Semester</th>
                        <th>Marks</th>
                    </tr>
                </thead>
                <tbody>
                    {students.map(student => (
                        <tr key={student.rollNo}>
                            <td>{student.rollNo}</td>
                            <td>{student.name}</td>
                            <td>{student.course}</td>
                            <td>{student.semester}</td>
                            <td>{student.marks}</td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}
export default App;
```



Part G: Create POST API

Now modify server.js to accept data from React.

server.js

```
const express = require("express");
const cors = require("cors");
const app = express();
const PORT = 5000;
// Middleware
app.use(cors());
app.use(express.json());
// Student data
let students = [
    {
        rollNo: 101,
        name: "Ayesha",
        course: "BSc IT",
        semester: 1,
        marks: 85
    },
    {
        rollNo: 102,
        name: "Priya",
        course: "BSc IT",
        semester: 1,
        marks: 90
    },
    {
        rollNo: 103,
        name: "Amit",
        course: "BSc IT",
        semester: 1,
        marks: 78
    }
];
// Home API
app.get("/", (req, res) => {
    res.send("Student REST API is running");
});
app.get("/api/students", (req, res) => {
    res.json(students);
});
app.post("/api/students", (req, res) => {
    const newStudent = req.body;
    students.push(newStudent);
    res.status(201).json({
        message: "Student added successfully",
        student: newStudent
    });
});
// Start Server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);

});
```

React Application

Open:

```
src/App.jsx
```

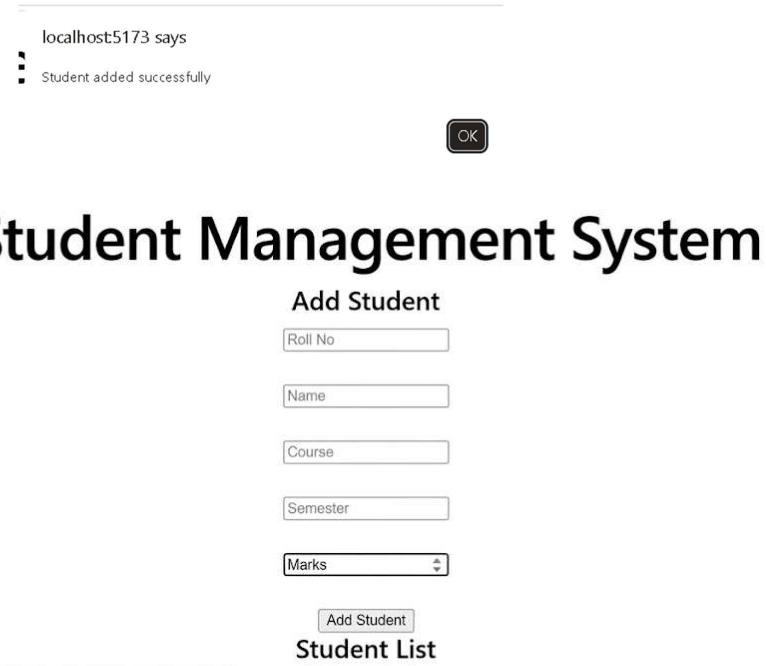
Use this code:

src/App.jsx

```
import { useEffect, useState } from "react";
function App() {
    // Student list
    const [students, setStudents] = useState([]);
    // Form fields
    const [rollNo, setRollNo] = useState("");
    const [name, setName] = useState("");
    const [course, setCourse] = useState("");
    const [semester, setSemester] = useState("");
    const [marks, setMarks] = useState("");
    const fetchStudents = () => {
        fetch("http://localhost:5000/api/students")
            .then(response => response.json())
            .then(data => {
                setStudents(data);
            })
            .catch(error => {
                console.error("Error:", error);
            });
    };
    // Fetch students when page loads
    useEffect(() => {
        fetchStudents();
    }, []);
    const addStudent = async (e) => {
        e.preventDefault();
        const newStudent = {
            rollNo: Number(rollNo),
            name: name,
            course: course,
            semester: Number(semester),
            marks: Number(marks)
        };
        try {
            const response = await fetch(
                "http://localhost:5000/api/students",
                {
                    method: "POST",
                    headers: {
                        "Content-Type": "application/json"
                    },
                    body: JSON.stringify(newStudent)
                }
            );
            const data = await response.json();
            alert(data.message);
            // Clear form
            setRollNo("");
            setName("");
            setCourse("");
            setSemester("");
            setMarks("");
            // Refresh student list
            fetchStudents();
        }
        catch (error) {
            console.error("Error:", error);
            alert("Error adding student");
        }
    };
    return (
```

```
        <div>
            <h1>Student Management System</h1>
            <h2>Add Student</h2>
            <form onSubmit={addStudent}>
                <input
                    type="number"
                    placeholder="Roll No"
                    value={rollNo}
                    onChange={(e) => setRollNo(e.target.value)}
                    required
                />
                <br /><br />
                <input
                    type="text"
                    placeholder="Name"
                    value={name}
                    onChange={(e) => setName(e.target.value)}
                    required
                />
                <br /><br />
                <input
                    type="text"
                    placeholder="Course"
                    value={course}
                    onChange={(e) => setCourse(e.target.value)}
                    required
                />
                <br /><br />
                <input
                    type="number"
                    placeholder="Semester"
                    value={semester}
                    onChange={(e) => setSemester(e.target.value)}
                    required
                />
                <br /><br />
                <input
                    type="number"
                    placeholder="Marks"
                    value={marks}
                    onChange={(e) => setMarks(e.target.value)}
                    required
                />
                <br /><br />
                <button type="submit">
                    Add Student
                </button>
            </form>
            <h2>Student List</h2>
            <table border="1" cellPadding="10">
                <thead>
                    <tr>
                        <th>Roll No</th>
                        <th>Name</th>
                        <th>Course</th>
                        <th>Semester</th>
                        <th>Marks</th>
                    </tr>
                </thead>
                <tbody>
                    {students.map(student => (
                        <tr key={student.rollNo}>
                            <td>{student.rollNo}</td>
                            <td>{student.name}</td>
                            <td>{student.course}</td>
                            <td>{student.semester}</td>
```

```
                            <td>{student.marks}</td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}
export default App;
```



Roll No	Name	Course	Semester	Marks
101	Ayesha	BSc IT	1	85
102	Priya	BSc IT	1	90
103	Amit	BSc IT	1	78
102	Piyush	BSc IT	1	88

Part H: Connect Express with MongoDB

Make sure MongoDB is running.

The MongoDB server normally uses:

```
mongodb://127.0.0.1:27017
```

Modify server.js:

server.js

```
const express = require("express");
const cors = require("cors");
const { MongoClient } = require("mongodb");
const app = express();
const PORT = 5000;
const mongoURL = "mongodb://127.0.0.1:27017";
const client = new MongoClient(mongoURL);
let studentsCollection;
app.use(cors());
app.use(express.json());
```

Part I: Create MongoDB Database and Collection

Add:

```
async function connectDB() {
    await client.connect();
    const db = client.db("CollegeDB");
    studentsCollection = db.collection("students");
    console.log("Connected to MongoDB");
}
connectDB();
```

The database is:

```
CollegeDB
```

The collection is:

```
students
```

MongoDB creates them when data is first stored if they do not already exist.

Part J: Create GET API Using MongoDB

Instead of returning hard-coded records, retrieve records from MongoDB.

```
app.get("/api/students", async (req, res) => {
    try {
        const students = await studentsCollection
            .find()
            .toArray();
        res.json(students);
    }
    catch (error) {
        res.status(500).json({
            message: "Error retrieving students",
            error: error.message
        });
    }
});
```

Important MongoDB Operation

```
find()
```

retrieves documents from the collection.

```
toArray()
```

converts the MongoDB cursor into an array.

Part K: Create POST API

Add:

```
app.post("/api/students", async (req, res) => {
    try {
        const student = {
            name: req.body.name,
            course: req.body.course,
            marks: req.body.marks
        };
        const result = await studentsCollection.insertOne(student);
        res.status(201).json({
            message: "Student added successfully",
            id: result.insertedId
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error inserting student",
            error: error.message
        });
    }
});
```

Part L: Send Data from React to Server

Now create a form in React.

Replace src/App.jsx with:

src/App.jsx

```
import { useEffect, useState } from "react";
function App() {
    const [students, setStudents] = useState([]);
    const [name, setName] = useState("");
    const [course, setCourse] = useState("");
    const [marks, setMarks] = useState("");
    // GET students
    const loadStudents = () => {
        fetch("http://localhost:5000/api/students")
            .then(response => response.json())
            .then(data => {
                setStudents(data);
            })
            .catch(error => {
                console.error(error);
            });
    };
    useEffect(() => {
        loadStudents();
    }, []);
    // POST student
    const addStudent = async (event) => {
        event.preventDefault();
        const student = {
            name: name,
            course: course,
```

```
            marks: Number(marks)
        };
        try {
            const response = await fetch(
                "http://localhost:5000/api/students",
                {
                    method: "POST",
                    headers: {
                        "Content-Type": "application/json"
                    },
                    body: JSON.stringify(student)
                }
            );
            const data = await response.json();
            console.log(data);
            setName("");
            setCourse("");
            setMarks("");
            loadStudents();
        }
        catch (error) {
            console.error("Error:", error);
        }
    };
    return (
        <div>
            <h1>Student Management</h1>
            <h2>Add Student</h2>
            <form onSubmit={addStudent}>
                <input
                    type="text"
                    placeholder="Student Name"
                    value={name}
                    onChange={(e) =>
                        setName(e.target.value)
                    }
                />
                <br /><br />
                <input
                    type="text"
                    placeholder="Course"
                    value={course}
                    onChange={(e) =>
                        setCourse(e.target.value)
                    }
                />
                <br /><br />
                <input
                    type="number"
                    placeholder="Marks"
                    value={marks}
                    onChange={(e) =>
                        setMarks(e.target.value)
                    }
                />
                <br /><br />
                <button type="submit">
                    Add Student
                </button>
            </form>
            <hr />
            <h2>Student List</h2>
            {students.map(student => (
                <div key={student._id}>
                    <h3>{student.name}</h3>
                    <p>
```

```
                        Course: {student.course}
                    </p>
                    <p>
                        Marks: {student.marks}
                    </p>
                    <hr />
                </div>
            ))}
        </div>
    );
}
export default App;
```

Part N: Test the Complete Application

Two servers need to be running.

Terminal 1 — Express Server

Go to the backend folder:

```
cd APIProject
```

Run:

```
node server.js
```

Terminal 2 — React Application

Go to the React project:

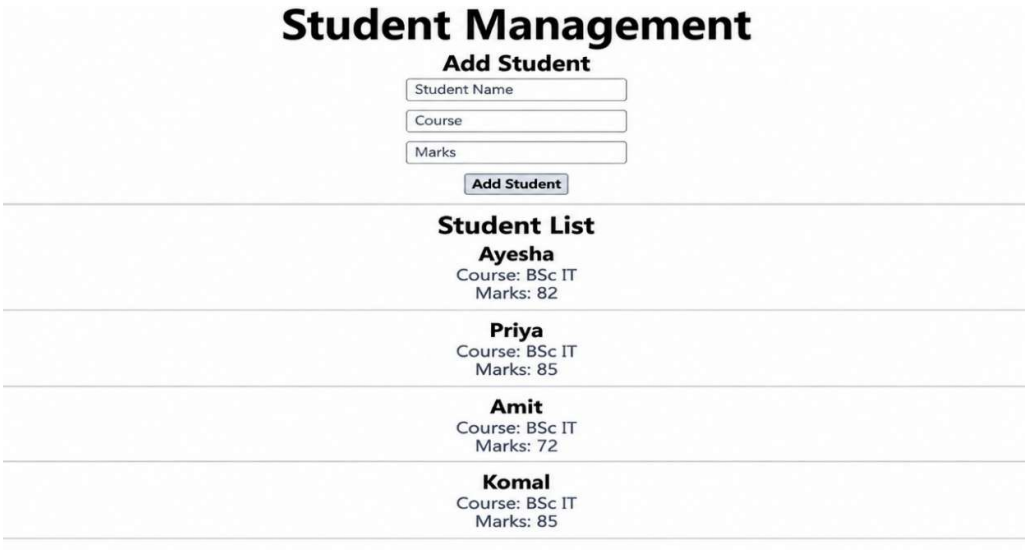
```
cd student-client
```

Run:

```
npm run dev
```

Open:

```
http://localhost:5173
```



Part O: Add a Student

Enter:

```
Name: Neha
Course: BSc IT
Marks: 92
```

Click:

Add Student

React sends:

```
POST /api/students
with:
{
    "name": "Neha",
    "course": "BSc IT",
    "marks": 92
}
```

Express receives the request and stores the document in MongoDB.

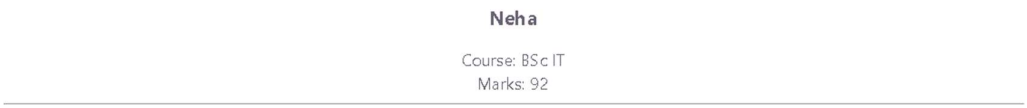
MongoDB document:

```
{
    "_id": "ObjectId(...)",
    "name": "Neha",
    "course": "BSc IT",
    "marks": 92
}
```

The React application then calls:

```
GET /api/students
```

and displays the updated list.



Part P: Verify Data in MongoDB

Open MongoDB Compass or mongosh.

In MongoDB Shell:

```
use CollegeDB
```

Display the collection:

```
show collections
```

Output:

```
students
```

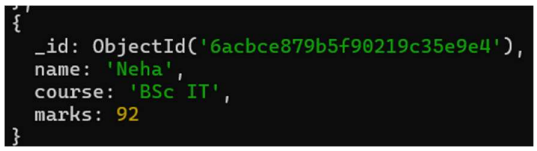
Display documents:

```
db.students.find()
```

Or:

```
db.students.find().pretty()
```

Example:



Part Q: Test APIs Using Postman

GET Request

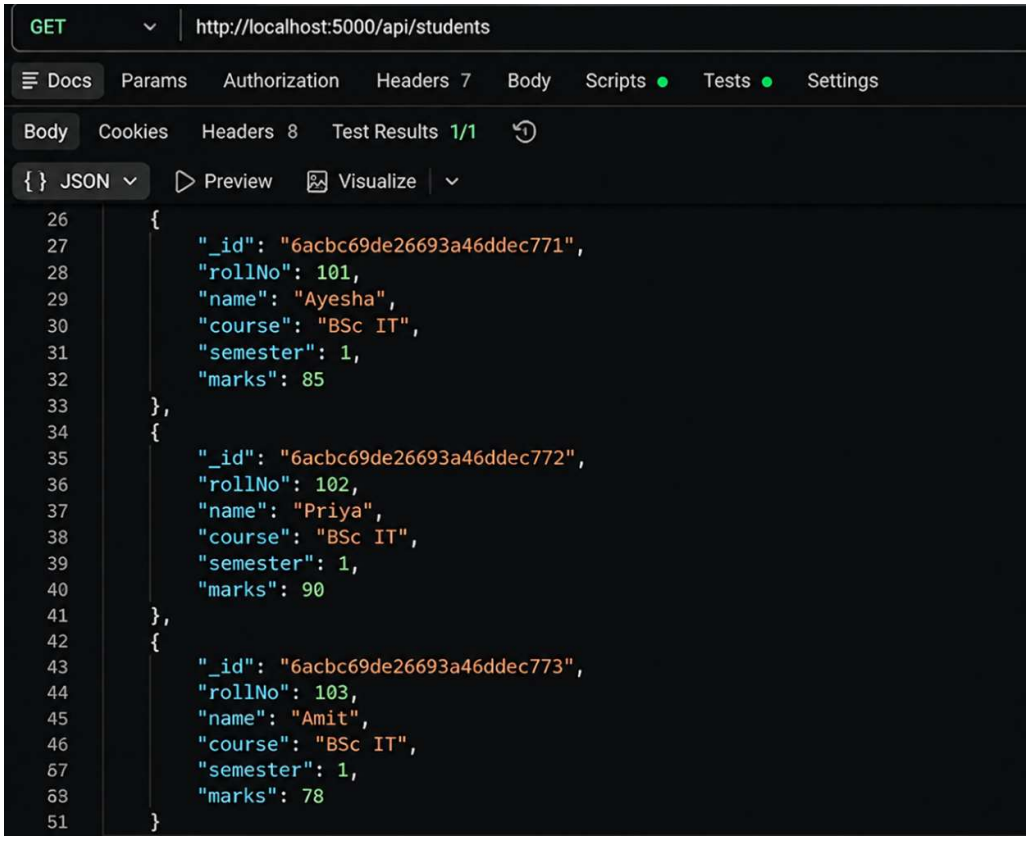
Method:

```
GET
```

URL:

```
http://localhost:5000/api/students
```

Click **Send**.



POST Request

Method:

```
POST
```

URL:

```
http://localhost:5000/api/students
```

Select:

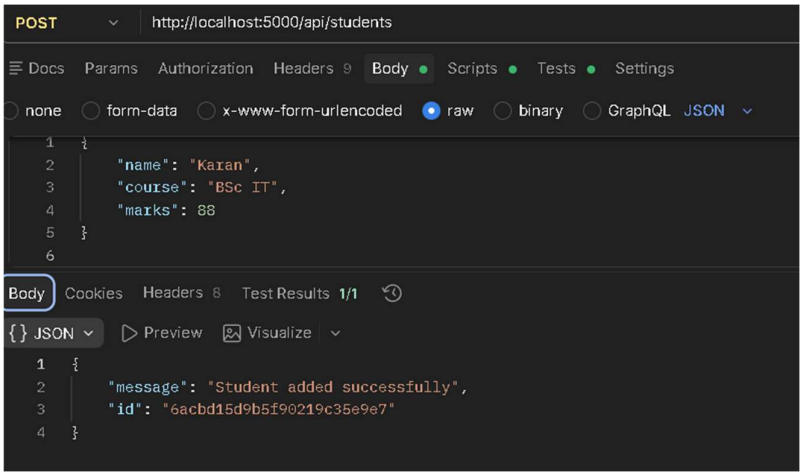
Body → raw → JSON

Enter:

```
{
    "name": "Karan",
    "course": "BSc IT",
    "marks": 88
}
```

Click **Send**.

Expected Response:



Result

Thus, a REST API was successfully created using Express.js. A GET API was implemented to retrieve student data, and a React application was used to fetch and display the data. A POST request was implemented to send student information from React to the Express server. The received data was stored in MongoDB and subsequently retrieved through the API and displayed on the React webpage.

Practical 9: User Data Management

Aim: To develop a full-stack user data management application using React.js, Node.js, Express.js, and MongoDB that allows users to add, retrieve, update, and delete user records.

(a) User Data Management using React, Express.js and MongoDB

Part A: Create the Express Backend

Step 1: Create Backend Project

Step 2: Install Required Packages

Install Express:

```
npm install express
```

Install MongoDB driver:

```
npm install mongodb
```

Install CORS:

```
npm install cors
```

Part B: Create Express Server

Create:

```
server.js
```

Enter:

server.js

```
const express = require("express");
const cors = require("cors");
const { MongoClient, ObjectId } = require("mongodb");
const app = express();
const PORT = 5000;
app.use(cors());
app.use(express.json());
const mongoURL = "mongodb://127.0.0.1:27017";
const client = new MongoClient(mongoURL);
let usersCollection;
```

Part C: Connect to MongoDB

Add the following function:

```
async function connectDB() {
    try {
        await client.connect();
        const database = client.db("UserDB");
        usersCollection = database.collection("users");
        console.log("MongoDB connected successfully");
    }
    catch (error) {
        console.error("MongoDB connection error:", error);
    }
}
connectDB();
```

The database is:

```
UserDB
```

The collection is:

```
users
```

MongoDB will create the database and collection when the first document is inserted if they do not already exist.

Part D: Create API to Add User

Add the following POST API:

```
app.post("/api/users", async (req, res) => {
    try {
        const user = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
        };
        const result = await usersCollection.insertOne(user);
        res.status(201).json({
            message: "User added successfully",
            id: result.insertedId
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error adding user",
            error: error.message
        });
    }
});
```

The data is stored as a MongoDB document.

Part E: Retrieve User Data

Create a GET API:

```
app.get("/api/users", async (req, res) => {
    try {
        const users = await usersCollection
            .find()
            .toArray();
        res.json(users);
    }
    catch (error) {
        res.status(500).json({
            message: "Error retrieving users",
            error: error.message
        });
    }
});
```

Part F: Update Existing User

Use the HTTP PUT method.

```
app.put("/api/users/:id", async (req, res) => {
    try {
        const id = req.params.id;
        const updatedUser = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
        };
        const result = await usersCollection.updateOne(
            { _id: new ObjectId(id) },
            { $set: updatedUser }
        );
        if (result.matchedCount === 0) {
            return res.status(404).json({
                message: "User not found"
            });
        }
        res.json({
            message: "User updated successfully"
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error updating user",
            error: error.message
        });
    }
});
```

Part G: Delete a User

Create the DELETE API:

```
app.delete("/api/users/:id", async (req, res) => {
    try {
        const id = req.params.id;
        const result = await usersCollection.deleteOne({
            _id: new ObjectId(id)
        });
        if (result.deletedCount === 0) {
            return res.status(404).json({
                message: "User not found"
            });
        }
        res.json({
            message: "User deleted successfully"
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error deleting user",
            error: error.message
        });
    }
});
```

The API deletes the selected document from MongoDB.

Part H: Start the Express Server

Add:

```
app.listen(PORT, () => {
    console.log(
        `Server running at http://localhost:${PORT}`
    );
});
```

Run:

```
node server.js
```

Expected Output:

```
MongoDB connected successfully
Server running at http://localhost:5000
```

Part I: Complete Backend Program

The complete server.js is:

server.js

```
const express = require("express");
const cors = require("cors");
const { MongoClient, ObjectId } = require("mongodb");
const app = express();
const PORT = 5000;
app.use(cors());
app.use(express.json());
const mongoURL = "mongodb://127.0.0.1:27017";
```

```
const client = new MongoClient(mongoURL);
let usersCollection;

// Connect to MongoDB
async function connectDB() {
    try {
        await client.connect();
        const database = client.db("UserDB");
        usersCollection = database.collection("users");
        console.log("MongoDB connected successfully");
    }
    catch (error) {
        console.error("MongoDB connection error:", error);
    }
}
connectDB();
// Home
app.get("/", (req, res) => {
    res.send("User Management API is running");
});
// CREATE
app.post("/api/users", async (req, res) => {
    try {
        const user = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
        };
        const result =
            await usersCollection.insertOne(user);
        res.status(201).json({
            message: "User added successfully",
            id: result.insertedId
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error adding user",
            error: error.message
        });
    }
});
// READ
app.get("/api/users", async (req, res) => {
    try {
        const users =
            await usersCollection.find().toArray();
        res.json(users);
    }
    catch (error) {
        res.status(500).json({
            message: "Error retrieving users",
            error: error.message
        });
    }
});
// UPDATE
app.put("/api/users/:id", async (req, res) => {
    try {
        const id = req.params.id;
        const updatedUser = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
```

```
        };
        const result =
            await usersCollection.updateOne(
                { _id: new ObjectId(id) },
                { $set: updatedUser }
            );
        if (result.matchedCount === 0) {
            return res.status(404).json({
                message: "User not found"
            });
        }
        res.json({
            message: "User updated successfully"
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error updating user",
            error: error.message
        });
    }
});
// DELETE
app.delete("/api/users/:id", async (req, res) => {
    try {
        const id = req.params.id;
        const result =
            await usersCollection.deleteOne({
                _id: new ObjectId(id)
            });
        if (result.deletedCount === 0) {
            return res.status(404).json({
                message: "User not found"
            });
        }
        res.json({
            message: "User deleted successfully"
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error deleting user",
            error: error.message
        });
    }
});
// Start server
app.listen(PORT, () => {
    console.log(
        `Server running at http://localhost:${PORT}`
    );
});
```

Part J: Create React Application

Open another terminal.

Create the React project:

```
npm create vite@latest user-client
```

Select:

```
Framework: React
Variant: JavaScript
```

Then:

```
cd user-client
npm install
```

Start the React application:

```
npm run dev
```

The application normally runs at:

```
http://localhost:5173
```

Part K: Create React User Form

Open:

```
src/App.jsx
```

Replace the existing code with:

src/App.jsx

```
import { useEffect, useState } from "react";

function App() {
    const [users, setUsers] = useState([]);
    const [name, setName] = useState("");
    const [email, setEmail] = useState("");
    const [phone, setPhone] = useState("");
    const [city, setCity] = useState("");
    const [editingId, setEditingId] = useState(null);
    // Retrieve users
    const loadUsers = async () => {
        try {
            const response =
                await fetch("http://localhost:5000/api/users");
            const data = await response.json();
            setUsers(data);
        }
        catch (error) {
            console.error("Error:", error);
        }
    };
    useEffect(() => {
        loadUsers();
    }, []);
    // Add or Update User
    const handleSubmit = async (event) => {
        event.preventDefault();
        const user = {
            name,
            email,
            phone,
            city
        };
        try {
```

```
            let response;
            if (editingId) {
                response = await fetch(
                    `http://localhost:5000/api/users/${editingId}`,
                    {
                        method: "PUT",
                        headers: {
                            "Content-Type": "application/json"
                        },
                        body: JSON.stringify(user)
                    }
                );
            }
            else {
                response = await fetch(
                    "http://localhost:5000/api/users",
                    {
                        method: "POST",
                        headers: {
                            "Content-Type": "application/json"
                        },
                        body: JSON.stringify(user)
                    }
                );
            }
            const data = await response.json();
            console.log(data);
            clearForm();
            loadUsers();
        }
        catch (error) {
            console.error("Error:", error);
        }
    };
    // Edit User
    const editUser = (user) => {
        setName(user.name);
        setEmail(user.email);
        setPhone(user.phone);
        setCity(user.city);
        setEditingId(user._id);
    };
    // Delete User
    const deleteUser = async (id) => {
        if (!window.confirm("Are you sure you want to delete this user?")) {
            return;
        }
        try {
            await fetch(
                `http://localhost:5000/api/users/${id}`,
                {
                    method: "DELETE"
                }
            );
            loadUsers();
        }
        catch (error) {
            console.error("Error:", error);
        }
    };
    // Clear form
    const clearForm = () => {
        setName("");
        setEmail("");
        setPhone("");
        setCity("");
```

```
        setEditingId(null);
    };
    return (
        <div>
            <h1>User Data Management</h1>
            <h2>
                {editingId ? "Update User" : "Add User"}
            </h2>
            <form onSubmit={handleSubmit}>
                <input
                    type="text"
                    placeholder="Enter Name"
                    value={name}
                    onChange={(e) =>
                        setName(e.target.value)
                    }
                    required
                />
                <br /><br />
                <input
                    type="email"
                    placeholder="Enter Email"
                    value={email}
                    onChange={(e) =>
                        setEmail(e.target.value)
                    }
                    required
                />
                <br /><br />
                <input
                    type="text"
                    placeholder="Enter Phone"
                    value={phone}
                    onChange={(e) =>
                        setPhone(e.target.value)
                    }
                    required
                />
                <br /><br />
                <input
                    type="text"
                    placeholder="Enter City"
                    value={city}
                    onChange={(e) =>
                        setCity(e.target.value)
                    }
                    required
                />
                <br /><br />
                <button type="submit">
                    {editingId ? "Update User" : "Add User"}
                </button>
                {editingId && (
                    <button
                        type="button"
                        onClick={clearForm}
                    >
                        Cancel
                    </button>
                )}
            </form>
            <hr />
            <h2>User List</h2>
            <table border="1" cellPadding="10">
                <thead>
                    <tr>
```

```
                        <th>Name</th>
                        <th>Email</th>
                        <th>Phone</th>
                        <th>City</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody>
                    {users.map(user => (
                        <tr key={user._id}>
                            <td>{user.name}</td>
                            <td>{user.email}</td>
                            <td>{user.phone}</td>
                            <td>{user.city}</td>
                            <td>
                                <button
                                    onClick={() =>
                                        editUser(user)
                                    }
                                >
                                    Edit
                                </button>
                                {" "}
                                <button
                                    onClick={() =>
                                        deleteUser(user._id)
                                    }
                                >
                                    Delete
                                </button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}
export default App;
```

Part Q: Test the Application

1. Start MongoDB

Ensure MongoDB is running.

For a Windows service:

```
net start MongoDB
```

Or open MongoDB through your installed MongoDB setup.

2. Start Backend

Inside the backend folder:

```
node server.js
```

Expected:

```
MongoDB connected successfully
Server running at http://localhost:5000
```

3. Start React

Inside the React project:

```
npm run dev
```

Open:

```
http://localhost:5173
```

Part R: Perform CRUD Operations

Operation 1: Add

Enter:

```
Name: Rahul
Email: rahul@example.com
Phone: 9876543210
City: Mumbai
```

Click:

Add User



Operation 2: View

The user appears automatically in the React table:

Name	Email	Phone	City
Rahul	rahul@example.com	9876543210	Mumbai

Operation 3: Update

Click:

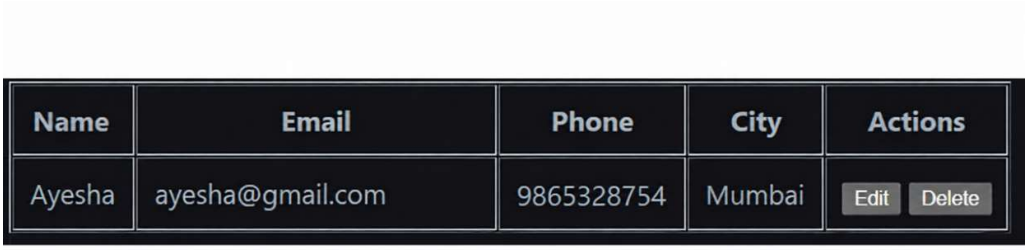
Edit

Change:

```
City: Pune
```

Click:

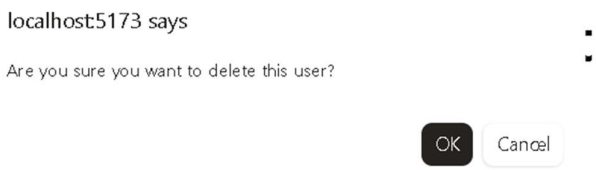
Update User



Operation 4: Delete

Click:

Delete



Part S: Verify Data in MongoDB

Open MongoDB Shell:

```
mongosh
```

Select the database:

```
use UserDB
```

Display collections:

```
show collections
```

Expected:

```
users
Display records:
db.users.find().pretty()
```

```
UserDB> db.users.find().pretty()
[
  {
    "_id": "6acbc69de26693a46ddec771",
    "name": "Ayesha",
    "email": "ayesha@gmail.com",
    "phone": "8394768265",
    "city": "Mumbai"
  },
  {
    "_id": "6acbe109b0af3ec8fca379b1",
    "name": "Piyush",
    "email": "piyush@example.com",
    "phone": "8974563217",
    "city": "Pune"
  }
]
```

Part T: Test APIs Using Postman

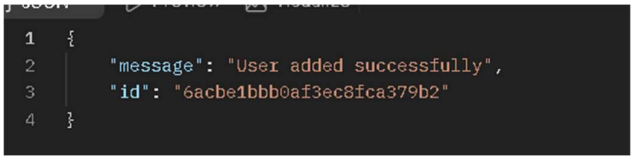
1. Add User

POST

```
http://localhost:5000/api/users
```

Body → raw → JSON:

```
{
    "name": "Priya",
    "email": "priya@example.com",
    "phone": "9876543211",
    "city": "Mumbai"
}
```



2. Retrieve Users

GET

```
http://localhost:5000/api/users
```

```
UserDB> db.users.find().pretty()
[
  {
    "_id": "6acbc69de26693a46ddec771",
    "name": "Ayesha",
    "email": "ayesha@gmail.com",
    "phone": "8394768265",
    "city": "Mumbai"
  },
  {
    "_id": "6acbe109b0af3ec8fca379b1",
    "name": "Piyush",
    "email": "piyush@example.com",
    "phone": "8974563217",
    "city": "Pune"
  }
]
```

3. Update User

PUT

```
http://localhost:5000/api/users/<USER_ID>
```

Body:

```
{
    "name": "Priya Sharma",
    "email": "priya@example.com",
    "phone": "9876543211",
    "city": "Thane"
}
```



4. Delete User

DELETE

```
http://localhost:5000/api/users/<USER_ID>
```



Result

Thus, a complete User Data Management application was successfully developed using React.js, Express.js, Node.js, and MongoDB. The application allows users to add, retrieve, update, and delete user records. The React interface dynamically displays the latest records after each CRUD operation without requiring a manual page refresh.

Practical 10: Full Stack Execution and Validation

Aim: To integrate a React frontend with a Node.js and Express.js backend, perform insert, update, and delete operations through the user interface, dynamically display MongoDB records, implement frontend form validation, test APIs using Postman, and verify the complete working of the full-stack application.

Full Stack Execution and Validation using React, Node.js, Express.js and MongoDB

Part A: Prepare the Backend

Create the backend project:

```
mkdir FullStackApp
cd FullStackApp
```

Initialize Node.js:

```
npm init -y
```

Install required packages:

```
npm install express mongodb cors
```

Create:

```
server.js
```

Part B: Create the Express Backend

Enter the following code in server.js:

server.js

```
const express = require("express");
const cors = require("cors");
const { MongoClient, ObjectId } = require("mongodb");
const app = express();
const PORT = 5000;
const mongoURL = "mongodb://127.0.0.1:27017";
const client = new MongoClient(mongoURL);
let usersCollection;
// Middleware
app.use(cors());
app.use(express.json());
// Connect MongoDB
async function connectDB() {
    try {
        await client.connect();
        const db = client.db("FullStackDB");
        usersCollection = db.collection("users");
        console.log("MongoDB connected successfully");
    }
    catch (error) {
        console.error("MongoDB connection error:", error);
    }
}
```

```
connectDB();

// Home route
app.get("/", (req, res) => {

    res.send("Full Stack API is running");

});

// GET - Retrieve users
app.get("/api/users", async (req, res) => {

    try {

        const users =
            await usersCollection.find().toArray();

        res.json(users);

    }
    catch (error) {

        res.status(500).json({
            message: "Error retrieving users"
        });

    }

});

// POST - Insert user
app.post("/api/users", async (req, res) => {

    try {

        const user = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
        };

        const result =
            await usersCollection.insertOne(user);

        res.status(201).json({
            message: "User added successfully",
            id: result.insertedId
        });

    }
    catch (error) {

        res.status(500).json({
            message: "Error adding user"
        });

    }

});

// PUT - Update user
app.put("/api/users/:id", async (req, res) => {

    try {
```

```
        const id = req.params.id;

        const updatedUser = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
        };

        const result =
            await usersCollection.updateOne(
                { _id: new ObjectId(id) },
                { $set: updatedUser }
            );

        if (result.matchedCount === 0) {

            return res.status(404).json({
                message: "User not found"
            });

        }

        res.json({
            message: "User updated successfully"
        });

    }
    catch (error) {

        res.status(500).json({
            message: "Error updating user"
        });

    }

});

// DELETE - Delete user
app.delete("/api/users/:id", async (req, res) => {

    try {

        const id = req.params.id;

        const result =
            await usersCollection.deleteOne({
                _id: new ObjectId(id)
            });

        if (result.deletedCount === 0) {

            return res.status(404).json({
                message: "User not found"
            });

        }

        res.json({
            message: "User deleted successfully"
        });

    }
    catch (error) {

        res.status(500).json({
```

```
            message: "Error deleting user"
        });

    }

});

// Start server
app.listen(PORT, () => {

    console.log(
        `Server running at http://localhost:${PORT}`
    );

});
```

Part C: Start MongoDB

Make sure MongoDB is running.

On Windows, if MongoDB is installed as a service:

```
net start MongoDB
```

You can verify MongoDB using:

```
mongosh
```

Then:

```
show dbs
```

Part D: Start the Backend

From the backend directory:

```
node server.js
```

Expected Output:

```
MongoDB connected successfully
Server running at http://localhost:5000
Test the home API in the browser:
http://localhost:5000/
```

Expected:

```
Full Stack API is running
```

Part E: Create the React Frontend

Open another terminal.

Create the React application using Vite:

```
npm create vite@latest frontend
```

Select:

```
Framework: React
Variant: JavaScript
```

Move into the project:

```
cd frontend
```

Install dependencies:

```
npm install
```

Start React:

```
npm run dev
```

The application normally runs at:

```
http://localhost:5173
```

Part F: Integrate React with Express

Open:

```
src/App.jsx
```

The React application will communicate with:

```
http://localhost:5000/api/users
```

The main communication is performed using the JavaScript fetch() function.

Part G: Create React Form with Validation

Replace src/App.jsx with:

src/App.jsx

```
import { useEffect, useState } from "react";
function App() {
    const [users, setUsers] = useState([]);
    const [name, setName] = useState("");
    const [email, setEmail] = useState("");
    const [phone, setPhone] = useState("");
    const [city, setCity] = useState("");
    const [editingId, setEditingId] = useState(null);
    const [errors, setErrors] = useState({});
    // Retrieve records
    const loadUsers = async () => {
        try {
            const response =
                await fetch("http://localhost:5000/api/users");
            const data = await response.json();
            setUsers(data);
        }
        catch (error) {
            console.error("Error:", error);
        }
    };
```

```
    useEffect(() => {
        loadUsers();
    }, []);
    // Frontend validation
    const validateForm = () => {
        const newErrors = {};
        if (name.trim() === "") {
            newErrors.name = "Name is required";
        }
        if (email.trim() === "") {
            newErrors.email = "Email is required";
        }
        else if (!/^\S+@\S+\.\S+$/.test(email)) {
            newErrors.email = "Enter a valid email";
        }
        if (phone.trim() === "") {
            newErrors.phone = "Phone number is required";
        }
        else if (!/^[0-9]{10}$/.test(phone)) {
            newErrors.phone =
                "Phone number must contain 10 digits";
        }
        if (city.trim() === "") {
            newErrors.city = "City is required";
        }
        setErrors(newErrors);
        return Object.keys(newErrors).length === 0;
    };
    // Insert / Update
    const handleSubmit = async (event) => {
        event.preventDefault();
        if (!validateForm()) {
            return;
        }
        const user = {
            name,
            email,
            phone,
            city
        };
        try {
            let response;
            if (editingId) {
                response = await fetch(
                    `http://localhost:5000/api/users/${editingId}`,
                    {
                        method: "PUT",
                        headers: {
                            "Content-Type": "application/json"
                        },
                        body: JSON.stringify(user)
                    }
                );
            }
            else {
                response = await fetch(
                    "http://localhost:5000/api/users",
                    {
                        method: "POST",
                        headers: {
                            "Content-Type": "application/json"
                        },
                        body: JSON.stringify(user)
                    }
                );
            }
```

```
            const data = await response.json();
            console.log(data);
            clearForm();
            loadUsers();
        }
        catch (error) {
            console.error("Error:", error);
        }
    };
    // Edit
    const editUser = (user) => {
        setName(user.name);
        setEmail(user.email);
        setPhone(user.phone);
        setCity(user.city);
        setEditingId(user._id);
        setErrors({});
    };
    // Delete
    const deleteUser = async (id) => {
        const confirmDelete =
            window.confirm(
                "Are you sure you want to delete this user?"
            );
        if (!confirmDelete) {
            return;
        }
        try {
            const response = await fetch(
                `http://localhost:5000/api/users/${id}`,
                {
                    method: "DELETE"
                }
            );
            const data = await response.json();
            console.log(data)
            loadUsers();
        }
        catch (error) {
            console.error("Error:", error);
        }

    };
    // Clear form
    const clearForm = () => {
        setName("");
        setEmail("");
        setPhone("");
        setCity("");
        setEditingId(null);
        setErrors({});
    };
    return (
        <div>
            <h1>User Management System</h1>
            <h2>
                {editingId
                    ? "Update User"
                    : "Add User"}
            </h2>
            <form onSubmit={handleSubmit}>
                <div>
                    <input
                        type="text"
                        placeholder="Enter Name"
                        value={name}
```

```
                        onChange={(e) =>
                            setName(e.target.value)
                        }
                    />
                    {errors.name && (
                        <p>{errors.name}</p>
                    )}
                </div>
                <br />
                <div>
                    <input
                        type="text"
                        placeholder="Enter Email"
                        value={email}
                        onChange={(e) =>
                            setEmail(e.target.value)
                        }
                    />
                    {errors.email && (
                        <p>{errors.email}</p>
                    )}
                </div>
                <br />
                <div>
                    <input
                        type="text"
                        placeholder="Enter Phone"
                        value={phone}
                        onChange={(e) =>
                            setPhone(e.target.value)
                        }
                    />
                    {errors.phone && (
                        <p>{errors.phone}</p>
                    )}
                </div>
                <br />
                <div>
                    <input
                        type="text"
                        placeholder="Enter City"
                        value={city}
                        onChange={(e) =>
                            setCity(e.target.value)
                        }
                    />
                    {errors.city && (
                        <p>{errors.city}</p>
                    )}
                </div>
                <br />
                <button type="submit">
                    {editingId
                        ? "Update User"
                        : "Add User"}
                </button>
                {editingId && (
                    <button
                        type="button"
                        onClick={clearForm}
                    >
                        Cancel
                    </button>
                )}
            </form>
            <hr />
```

```
            <h2>Users from MongoDB</h2>

            <table border="1" cellPadding="10">
                <thead>
                    <tr>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Phone</th>
                        <th>City</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody>
                    {users.map(user => (
                        <tr key={user._id}>
                            <td>{user.name}</td>
                            <td>{user.email}</td>
                            <td>{user.phone}</td>
                            <td>{user.city}</td>
                            <td>
                                <button
                                    onClick={() =>
                                        editUser(user)
                                    }
                                >
                                    Edit
                                </button>
                                {" "}
                                <button
                                    onClick={() =>
                                        deleteUser(user._id)
                                    }
                                >
                                    Delete
                                </button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}
export default App;
```

Part H: Understand Frontend Validation

The validateForm() function checks whether the entered information is valid.

1. Name Validation

```
if (name.trim() === "") {
    newErrors.name = "Name is required";
}
```

The name cannot be empty.



2. Email Validation

```
else if (!/^\S+@\S+\.\S+$/.test(email)) {
    newErrors.email = "Enter a valid email";
}
```

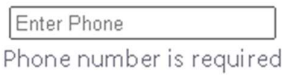
This checks the basic email format.

For example:



3. Phone Validation

```
else if (!/^[0-9]{10}$/.test(phone)) {
    newErrors.phone =
        "Phone number must contain 10 digits";
}
```



4. City Validation

```
if (city.trim() === "") {
    newErrors.city = "City is required";
}
```



Part I: Dynamic Database Display

The function:

```
const loadUsers = async () => {
    const response =
        await fetch("http://localhost:5000/api/users");
    const data = await response.json();
    setUsers(data);
};
```

retrieves the latest records from MongoDB through the Express API.

React displays the records using:

```
users.map(user => (
    ...
))
```

Part J: Test Insert Operation through UI

Open:

```
http://localhost:5173
```

Enter:

```
Name: Rahul
Email: rahul@gmail.com
Phone: 9876543210
City: Mumbai
```

Click:

Add User

Name	Email	Phone	City	Actions
Mantasha	mantasha@gmail.com	8369407680	Mumbai	Edit Delete
Ayesha	ayesha@gmail.com	9305998978	Bangalore	Edit Delete
Saamiya	Saamiya@gmail.com	3694689469	Pune	Edit Delete

Part K: Test Update Operation through UI

Click **Edit** beside an existing user.

The user's details will be loaded into the form.

For example:

```
Name: Rahul
Email: rahul@gmail.com
Phone: 9876543210
City: Pune
```

Click:

Update User

The React application sends:

```
PUT /api/users/:id
```

Express updates MongoDB using:

```
updateOne()
```

The updated record is then displayed in React.



Part L: Test Delete Operation through UI

Click **Delete** beside a user.

A confirmation message appears:

Are you sure you want to delete this user?

Click **OK**.

React sends:

```
DELETE /api/users/:id
```

Express executes:

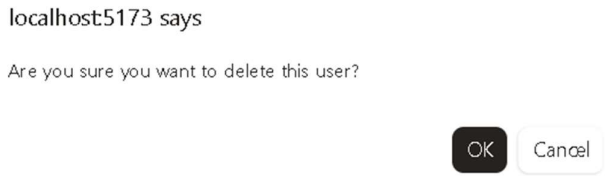
```
deleteOne()
```

The record is deleted from MongoDB.

React then calls:

```
loadUsers();
```

and the deleted record disappears from the webpage.



Part M: Test Frontend Validation

Test 1: Empty Form

Click **Add User** without entering any information.

Expected messages:

```
Name is required
Email is required
Phone number is required
City is required
```



Test 2: Invalid Email

Enter:

```
student@gmail
```

Expected:



Test 3: Invalid Phone

Enter:

```
12345
```

Expected:



Test 4: Valid Information

Enter:

```
Name: Priya
Email: priya@gmail.com
Phone: 9876543211
City: Mumbai
```

The validation should succeed and the record should be inserted.

Part N: Test APIs Using Postman

All APIs should be independently tested using Postman.

1. Test GET API

Method:

```
GET
```

URL:

```
http://localhost:5000/api/users
```

Click **Send**.



2. Test POST API

Method:

```
POST
```

URL:

```
http://localhost:5000/api/users
```

Select:

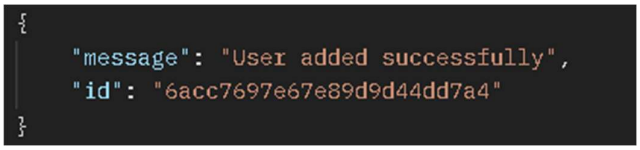
Body → raw → JSON

Enter:

```
{
    "name": "Amit",
    "email": "amit@gmail.com",
    "phone": "9876543212",
    "city": "Nashik"
}
```

Click **Send**.

Expected Response:



3. Test PUT API

Copy the _id of an existing user.

Method:

```
PUT
```

URL:

```
http://localhost:5000/api/users/<USER_ID>
```

Body:

```
{
    "name": "Amit Sharma",
    "email": "amit@gmail.com",
    "phone": "9876543212",
    "city": "Pune"
}
```

Expected Response:



4. Test DELETE API

Method:

```
DELETE
```

URL:

```
http://localhost:5000/api/users/<USER_ID>
```

Expected Response:

```
{
    "message": "User deleted successfully
}
```

Part O: Verify MongoDB

Open MongoDB Shell:

```
mongosh
```

Select the database:

```
use FullStackDB
```

Display collections:

```
show collections
```

Expected:

```
users
Display records:
db.users.find().pretty()
```

```
UserDB> db.users.find().pretty()
[
  {
    _id: ObjectId('6acbe0ecb0af3ec8fca379b0'),
    name: 'Ayesha',
    email: 'ayesha@gmail.com',
    phone: '8394768265',
    city: 'Mumbai'
  },
  {
    _id: ObjectId('6acbe109b0af3ec8fca379b1'),
    name: 'Saamiya',
    email: 'Saamiya@gmail.com',
    phone: '8974563217',
    city: 'Pune'
  }
]
```

Result

Thus, the React frontend was successfully integrated with the Node.js, Express, and MongoDB backend. CRUD operations, dynamic data display, form validation, and REST API testing using Postman were completed and verified successfully.